LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CU6051NI Artificial Intelligence

Individual Coursework

Submission: Final Submission

Academic Semester: Autumn Semester 2025

Credit: 15 credit semester long module

Student Name: Digdarshan Bhattarai

London Met ID: 23049051

College ID: NP01CP4A230320

Assignment Due Date: 21/01/2026.

Assignment Submission Date: 20/01/2026

Submitted To: Er. Roshan Shrestha

GitHub Link	*https://github.com/DigdarshanB/mushroom-edibility-classification*

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

Table of Figures

Table of Tables

1. Introduction

Mushrooms are widely consumed across the globe, loved for being a delicious, healthy, food source. However, determining which ones to eat and which ones to avoid is a big hassle. Many varieties of poisonous species appear nearly the same shape, colour, and size as the ones that can be consumed, and thus cannot be detected by merely visual inspection. Even a small mistake during identification can result into severe illness or even death.

Figure 1: Commonly eaten around Nepal, Oyster Mushroom (Kanne Chyau) and Poisonous Omphalotus olivascens, often misidentified.

The conventional methods of identifying mushrooms have relied on human experience, reference books, or visual examination of physical characteristics. Such approaches are generally subjective, slow and inaccurate. Fortunately, there is an increase in the availability of structured biological data, which opens the prospects of applying computational methods in enhancing accuracy and consistency in the process of classifying mushrooms.

As the amount of structured biological data increases, there is an excellent chance to use AI methods to assist in classifying mushrooms. AI systems are ideal because they can compare data in large amounts in a timely and objective manner and are therefore suitable in safety-sensitive decision-making tasks, such as determining the edibility of mushrooms.

1.1 Aim and Objectives of the Project

The overall goal of the project is to use and assess the suitability of supervised machine learning algorithms to determine whether mushrooms are edible or poisonous, based on an already existing dataset and the objectives are:

- To prepare and investigate the mushroom dataset to learn its features and adaptability to supervised classification.
- To use chosen supervised machine learning algorithms to the mushroom data to make the edibility classification.
- To compare the performance of the algorithms on the standard classification metrics to figure out which approach is most appropriate in this issue.

1.2 Overview of the Dataset and Problem Formulation

In this project we are addressing the problem of mushroom edibility and solving it as a binary classification problem - each mushroom is assigned with either edible or poisonous tag

The data is taken out of the UCI Machine Learning Repository a famous go-to resource in AI research benchmark data, which currently has and maintains 688 datasets, serving students of machine learning and AI. (UCI, 2025)

The dataset consists of 8,124 rows which consists various kinds of mushrooms and 22 columns, having attributes such as:

- Cap shape
- Cap colour
- Odour
- Gill size
- Gill colour
- Habitat

Each mushroom has been classified as either as edible or poisonous, hence the data samples are excellent at supervised learning. The attributes are diverse and rich which enables machine-learning models to identify subtle patterns.

1.3 AI Concepts Used

a) Supervised Machine Learning

Supervised ML, simply put means training a model on known dataset to predict outcomes for unknown data. First step is to train a model, then it can be used to predict new inputs (Verma, 2025).

Supervised learning is especially suitable to this problem as the correct classifications are already present in historical data. The model learns the relationship of feature and outcome and predicts correctly on new samples of the mushrooms, which are not visible.

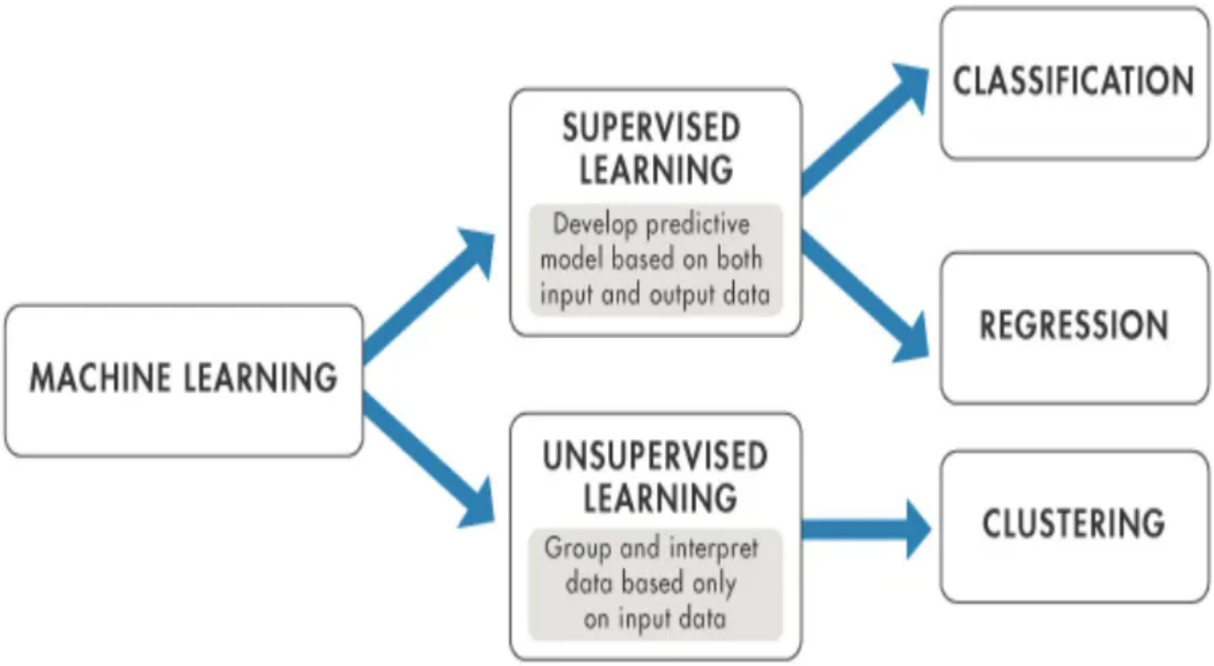

Figure 2: Types of Machine Learning (Verma, 2025)

1.4 Classification Algorithms

In this experiment, we have used and compared various supervised classification algorithms to determine which one is best used to classify a mushroom as being edible based on its categorical features. The algorithms used are:

a) Logistic Regression

Logistic Regression is used to predict binary outcomes.

b) Naïve Bayes

Naive Bayes predicts probability distribution over a set of classes.

c) Random Forest

Random Forest combines a collection of decision trees and is used to boost accuracy and reduce overfitting.

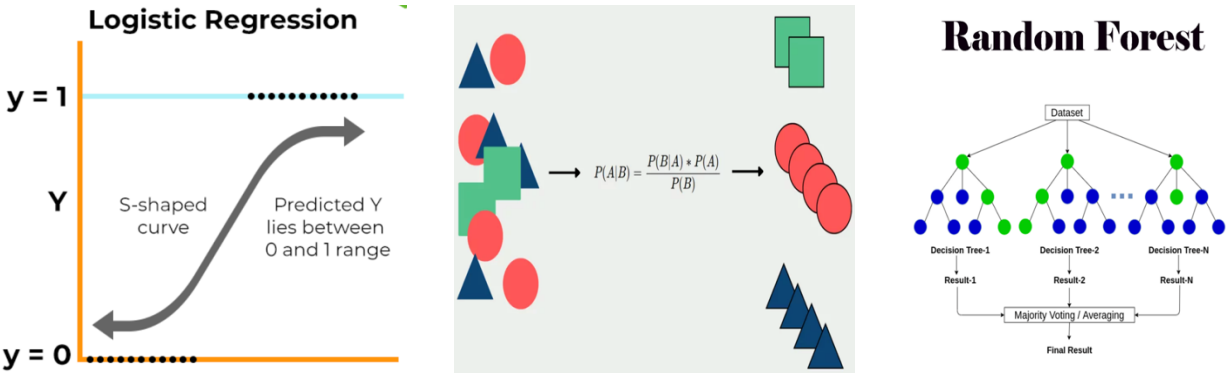

Figure 3: Visual overview of Logistic Regression, Naive Bayes, and Random Forest.

2. Background

2.1 Problem Context: Why Mushroom Identification is Hard

Correct identification of mushroom is crucial task because some deadly species resemble edible ones in shape, colour, and overall appearance closely just like the example above. People used to rely on expert knowledge, books and manuals which is both time consuming and difficult to apply consistently, especially for beginners.

The difficulty of this issue is especially the fact that this property is not defined by one obvious characteristic that makes the product edible. The own description of the dataset (and its UCI description) clearly points out that no single simple rule can be considered reliable to determine the mushroom edibility: rather, the choice relies on the combination of various traits. (UCI ML Repository, 1987) .

This is precisely what kind of environment supervised learning might come in: a model can gain the ability to learn many relations between many attributes at once and generalize them.

2.2 Dataset Background and Origin

The dataset used in this coursework is the popular UCI mushroom dataset (Agaricus and Lepiota). It includes the description of hypothetical Herbs which represent 23 species of the gilled mushrooms of the family Agaricus and Lepiota which had been first present in The Audubon Society Field Guide to North American Mushrooms (1981).

Instances: 8,124

The predictor attributes include: 22 (all nominal/categorical).

Label: target: It is edible (e) or poisonous (p).

Distribution of class:

- edible = 4,208 (51.8%)
- poisonous = 3,916 (48.2%)

In the data set, it is also mentioned that species may be definitely edible, definitely poisonous, or unknown/not recommended, and the unknown/not recommended category is grouped in with the toxic category. It becomes significant as it influences us to perceive the label of poisonous in a certain way (this also involves cases of do not eat, but not necessarily being medically poisonous).

2.3 Research on Mushroom Classification

Mushroom Edibility prediction has been studied by researchers mainly as supervised classification problem, with the goal to predict edible vs unsafe mushroom from recorded traits. The UCI mushroom dataset is used for this purpose because it is stated that there is not any particular rule for finding out edibility, meaning many features should be considered together (UCI ML Repository, 1987) .

2.3.1 How mushrooms are identified usually

Traditionally, mushrooms have been characterised by observable characteristics including cap shape/colour, gill features, stalk features, spore print colour, habitat and smell. This method is effective with experts, though may be challenging with beginners since numerous edible and toxic mushrooms are similar. This is why scientists have considered the application of AI to help the process of the decision making based on learning patterns of the recorded mushroom characteristics.

2.3.2 Why AI is used for this problem

The edibility of mushrooms is an AI study that focuses on mushroom edibility as a supervised classification problem: in the training procedure, examples that specify the correct label (edible or poisonous) are provided. This is helpful since it is a combination of traits that classify edibility rather than an individual property. The description of the UCI data also indicates that it cannot be determined whether the item is edible using a single simple rule and because of this, multi-feature machine learning models can be used.

2.4 Review of Existing Work on Mushroom Edibility Prediction

a) Prior Use of Random Forest/Decision Trees on Mushroom Datasets

Tree-based methods are widely used because they represent step by step decisions using categorical features. To put it into perspective, if the odour of mushroom is x, then y, and so on.

In a study which compares Random Forest and REP Tree on the UCI mushroom dataset, the study reports high performance with Random Forest achieving 100% accuracy and REP Tree achieving slightly lover but still high results.

This suggests that for this UCI dataset, tree-based models often perform well because they capture interactions between traits. That said, the scores should still be interpreted carefully as real-world identification can involve uncertain or missing data too (Nepal Journals Online, 2022).

Nepal Journal of Mathematical Sciences (NJMS)
ISSN: 2738-9928 (online), 2738-9812 (print)
Vol. 3, No. 1, 2022 (February): 111-116
DOI: https://doi.org/10.3126/njmathsci.v3i1.44130

Research Article
Received Date: December 25, 2021
Accepted Date: February 24, 2022
Published Date: February 28, 2022

Mushroom Classification using Random Forest and REP Tree Classifiers

Nawaraj Paudel[1] and Jagdish Bhatta[2]

[1,2]*Central Department of Computer Science and IT, Tribhuvan University, Kathmandu, Nepal*
Email: [1]*nawarajpaudel@cdcsit.edu.np*, [2]*jagdish@cdcsit.edu.np*
Corresponding Author: Jagdish Bhatta

Abstract: *Mushroom is a reproductive structure produced by some fungi that has a high level of protein and a rich source of vitamin B. It aids in the prevention of cancer, weight loss, and immune system enhancement. There are numerous thousands of mushroom species within the world and a few are edible and a few are noxious due to noteworthy poisons on them. Hence, it is a vital errand to distinguish between edible and harmful mushrooms. This paper focuses on comparing the performance of two tree-based classification algorithms, Random Forest and Reduced Error Pruning (REP) Tree, for the classification of edible and poisonous mushrooms. In this paper, mushroom dataset from UCI machine learning repository has been classified using Random Forest and REP Tree classifiers. The evaluation of these two algorithms using accuracy, precision, recall and F-measure shows that the Random Forest outperforms REP Tree algorithm with value of 100% for accuracy, precision, recall and F- measure. The performance of Random Forest is 100% and is better with respect to REP Tree classifier.*

Keywords: *Mushroom Dataset, Random Forest, REP Tree, 10-fold cross-validation Confusion Matrix.*

Figure 4: Random Forest vs REP Tree (Poudel, Bhatta, 2022)

4. Experiments and Results

The two classification algorithms were executed on the mushroom dataset using 10-folds cross-validation for the classification of mushrooms based on their class labels. The table below shows confusion matrix of the classification report that has been obtained after testing Random Forest algorithm.

Table 1 Confusion Matrix of Random Forest Algorithm

		Predicted class		
		poisonous	edible	Total
Actual Class	Poisonous	3916	0	3916
	Edible	0	4208	4208
	Total	3916	4208	8124

The table below shows confusion matrix of the classification report that has been obtained after testing REPT Tree algorithm.

Table 2 Confusion Matrix of REP Tree Algorithm

		Predicted class		
		poisonous	edible	Total
Actual Class	Poisonous	3916	0	3916
	Edible	2	4206	4208
	Total	3918	4206	8124

Based on the classification reports shown in Table 1 and Table 2, the calculated summary performance result for the comparison of two algorithms applied on mushroom dataset is shown in the table below. The accuracy, precision, recall and F-measure value are shown is the average of precision, recall and F-measure for both categories.

Table 3 Performance result of two algorithms

Algorithm	Accuracy	Precision	Recall	F-measure
Random Forest	100%	100%	100%	100%
REP Tree	99.98%	99.95%	100%	99.97%

It is clearly seen that the accuracy, precision, recall, and F-measure values of Random Forest is 100% and that of REP Tree is 99.98%, 99.95%, 100%, and 99.97% respectively.

Figure 5: Result of the study (Paudel, Bhatta, 2022)

Advantages:

- Can interact with categorical features well.
- Can often attain extremely high benchmark accuracy on this dataset.
- Offer feature-importance style methods of information (in particular, with Random Forest).

Disadvantages:

- Random Forest is less interpretable than a single decision tree.
- Model behaviour may seem too good to be true on benchmark data and should be carefully examined.
- The behaviour of the models can vary.

b) Prior Use of Naïve Bayes/ Logistic Regression on Mushroom Datasets

Some works use Naïve Bayes and Logistic Regression, and these models are fast, easy to compare and provide a 'reference level' of performance which is why these models are very useful.

A study by Sulistianingsih and Martono (2025) that explains several supervised learning models that are used to classify mushrooms edibility using the UCI Mushroom dataset (8,124 instances and 22 attributes). Both Random Forest and one of their Stacking models are found to have 100% accuracy whereas Naive Bayes has significantly lower-performance (59.8% accuracy) due to the strong assumption of conditional independence by Naivete Bayes that the authors claim is very poor in capturing the interactional nature of mushroom traits.

In general, this work justifies the adoption of Logistic Regression and Naive Bayes as benchmarks and recommends the adoption of tree based / ensemble models to this field since they can represent the interactions between features. (ResearchGate, 2025)

J-INTECH (Journal of Information and Technology)
Accredited Sinta 4 Ministry of Higher Education, Science and Technology Republic of Indonesia SK No. 10/C/C3/DT.05.00/2025
E-ISSN: 2580-720X || P-ISSN: 2303-1425

J-INTECH Journal of Information and Technology

Analysis of the Effectiveness of Traditional and Ensemble Machine Learning Models for Mushroom Classification

Neny Sulistianingsih[1*], Galih Hendo Martono[2]

[1,2]*Departement of Computer Science, Master Program, Universitas Bumigora, Ismail Marzuki St. Mataram, Indonesia*

Keywords
Bagging; Ensemble Learning; K-Nearest Neighbors; Mushroom Classification; Random Forest; Stacking; Voting Classifier

***Corresponding Author:**
neny.sulistianingsih@universitasbumigora.ac.id

Abstract
The classification of edible versus poisonous mushrooms presents a critical challenge in the domains of applied biology and public health, particularly due to the serious implications of misidentification. This research employs the UCI Mushroom Dataset to evaluate and compare the effectiveness of several machine learning models, including traditional algorithms like Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, K-Nearest Neighbors and Naïve Bayes, as well as advanced ensemble techniques such as Stacking and Voting Classifier. Notably, both Random Forest and Stacking achieved flawless accuracy, reaching 100%, underscoring the high predictive capacity of these models in complex categorical scenarios. Conversely, Naïve Bayes exhibited significantly weaker performance—achieving only 59.8% accuracy—likely due to its underlying assumption of feature independence, which does not hold for this dataset. The ensemble learning approaches, including the combination of Stacking and Bagging, not only preserved but also enhanced model robustness and generalization. These methods effectively leverage the complementary strengths of individual learners to yield more accurate and stable predictions while mitigating overfitting risks. Comparative analysis with previous research confirms the consistency of these findings and reinforces the viability of ensemble strategies for handling intricate classification tasks. Overall, this study highlights the importance of algorithm selection tailored to data characteristics and supports the use of ensemble learning to boost predictive reliability.

Figure 6: Traditional vs Ensemble models (Sulistianingsih & Martono, 2025)

Table 5. Classification Results with Traditional Methods

Model	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.841	0.842	0.841	0.842	0.913
Decision Tree	0.998	0.998	0.998	0.998	0.998
Support Vector Machine	0.995	0.995	0.995	0.995	0.999
K-Nearest Neighbors	1.000	1.000	1.000	1.000	1.000
Naive Bayes	0.598	0.787	0.598	0.550	0.835

Figure 7: Result of Baselines vs ensembles (Sulistianingsih & Martono, 2025)

Advantages:

- Logistic Regression is easy and easily explained as a baseline.
- Naive Bayes is fast and is usually used to form the basis of categorical classification.
- Both yield a clear baseline with which the ensembles can be evaluated as adding meaningful contributions.

Disadvantages:

- A Logistic Regression can fare poorly when the boundary of the decision is far from linear.
- Naive Bayes can fail badly when the independence properties are broken.
- It has been found that this is the case in this dataset.

c) Prior Use of Multiple Classifiers on Mushroom Datasets

Besides individual model comparisons, several studies also compare multiple supervised classifiers on the UCI mushroom dataset. The feature selection employed by Tank (2021) is the Principal Component Analysis (PCA) and various models such as Logistic Regression, Decision Tree, KNN, SVM, Naive Bayes, and Random Forest were tested. The paper provides a comparison of model performance in ROC-based performance measure, which justifies the notion that prediction of mushroom edibility is generally considered a benchmark classification task where many algorithms are assumed to be tested simultaneously and then one algorithm decided to be applied.

MINIMAL DECISION RULES BASED ON THE APRIORI ALGORITHM †

María C. FERNÁNDEZ*, Ernestina MENASALVAS*, Óscar MARBÁN*
José M. PEÑA**, Socorro MILLÁN***

Based on rough set theory many algorithms for rules extraction from data have been proposed. Decision rules can be obtained directly from a database. Some condition values may be unnecessary in a decision rule produced directly from the database. Such values can then be eliminated to create a more comprehensible (minimal) rule. Most of the algorithms that have been proposed to calculate minimal rules are based on rough set theory or machine learning. In our approach, in a post-processing stage, we apply the Apriori algorithm to reduce the decision rules obtained through rough sets. The set of dependencies thus obtained will help us discover irrelevant attribute values.

Keywords: rough sets, rough dependencies, association rules, Apriori algorithm, minimal decision rules

Figure 8:Use of Multiple Classifiers on Mushroom Datasets (Fernandez, 2001)

Advantages:

- A fair big picture comparison.
- Facilitates easier justification of results due to benchmarking of performance both against simple control baselines and stronger models
- Allows to understand what algorithms are most consistent across evaluation measures.

Disadvantages:

- Preprocessing options can bias results (e.g. the choice of encoding algorithm and the use of PCA)
- Comparison can be unreliable with models not tuned equally
- Interpretability may be compromised by dimensionality-reduction techniques such as PCA,

2.5 Key Takeaways from Existing Works

Much of the previous research indicates that models of mushroom edibility can acquire clear patterns of choices based on categorical characteristics. Various studies identify a consistent theme which is that tree-based methods especially the random forest are frequently able to perform to an extremely high benchmark score on the UCI mushroom data. An example by Paudel and Bhatta (2022), where they contrasted Random Forest with a pruned decision tree (REP Tree) had 100 percent accuracy with Random Forest with a REP Tree only slightly lower, indicating that tree-based logic is well-polarized with the interactions between the mushroom properties and the forecast of edibility.

Simultaneously, Logistic Regression and Naive Bayes are often used as a baseline model in studies. These baselines assist in interpreting the results, in that, they indicate the presence of meaningful improvement in the more sophisticated procedures. According to Sulistianingsih and Martono, (2025) nothing can be more accurate than Random Forest/stacking, and much less accurate than Naïve Bayes, which confirms the fact that mushroom characteristics tend to be interactive and that models based on simplistic assumptions are possibly not appropriate.

3. Solutions

3.1 Proposed Solution

The proposed solution for the project is a supervised ML pipeline that predicts whether a mushroom is edible or not based on recorded categorical attributes. The solution is designed to be implemented in Jupyter Notebook using Python as the programming language. The solution follows a structured workflow to evaluate every step fairly.

Three classification algorithms have been used – Logistic Regression, Naïve Bayes, and Random Forest for comparative analysis. These models present different levels of complexity. Logistic Regression provides a basic linear baseline for the project. Naïve Bayes is probability-based baseline which is best suited for categorical data and Random Forest is an ensemble model that can capture feature interactions. The final model recommendation will be based on evaluations of the train/test data.

3.2 System Workflow

1. **Load and Structure Data**: .data file is loaded into a table and feature names are applied from .names file.
2. **Data Cleaning**: '?' values are replaced with 'missing' and uninformative columns are dropped.
3. **Quality Checks and EDA**: Missing, duplication counts, class distribution plot.
4. **Split the dataset:** Dataset is divided into training and testing sets.
5. **Check For Leakage:** Check if train/test row overlap.
6. **Setup CV:** Setup StratifiedKFold.
7. **One-Hot Encoding:** Build pipelines.
8. **Model Training and hyperparameter tuning**: Train Logistic Regression, Naïve Bayes, and Random Forest on the training dataset and tune it with GridSearchCV.
9. **Compare Models**: Compare models using bar graphs, results table.
10. **Prediction**: Use trained models to predict class labels on the test set (100 rows).
11. **Evaluation and Comparison**: Compare models using standard classification metrics and justify the most suitable model.
12. **CLI**: Create CLI to predict Edible/Poisonous

3.3 Pseudocode for the Overall Project

INPUT: Dataset, user feature codes (CLI).

OUTPUT: tuned models + best estimators, evaluation metrics/plots, (CLI) Edible/Poisonous

START

1. DEFINE COLUMNS
2. LOAD dataset into df
3. CREATE df_clean
4. REPLACE '?' with 'missing'
5. DROP constant categories
6. DISPLAY basic checks (shape/info, missing counts, duplicates, class counts, class plot)
7. SET X = df_clean without class
8. SET y = class
9. SPLIT into x_train, x_test, y_train, y_test
10. CHECK if train/test row overlap
11. CREATE stratified K-fold CV
12. SET scoring to recall of class p
13. CREATE one-hot encoder
14. For each model in {Logistic Regression, Bernoulli Naïve Bayes, Random Forest}:
    - Build pipeline = one-hot encoder + model
    - Run grid search with CV on training set
    - Select best estimator
    - Predict on test set
    - Compute metrics (accuracy, precision(p), recall(p), f1(p))
    - Plot confusion matrix

15. CREATE results table
16. PLOT grouped bar chart (accuracy, precision(p), recall(p))
17. SAMPLE 100 rows
18. PREDICT 100 rows using chosen the best model.
19. COMPARE predicted vs actual
20. CLI:

- LOAD saved best pipeline
- Repeat: read user codes → replace ? with missing → build one-row input → predict → print Edible/Poisonous → exit when requested

END

3.4 Pseudocode for Each Algorithm

3.4.1 Pseudocode – Logistic Regression

START

1. USE x_train, y_train, x_test, y_test
2. CREATE one-hot encoder
3. CREATE Logistic Regression model
4. COMBINE encoder + model into one pipeline
5. DEFINE parameter options (C, penalty)
6. RUN grid search with stratified CV using recall
7. SELECT best LR pipeline
8. PREDICT on x_test
9. COMPUTE accuracy, precision(p), recall(p), f1(p)
10. GENERATE confusion matrix

END

3.4.2 Pseudocode – Naïve Bayes

START

1. USE x_train, y_train, x_test, y_test

2. CREATE one-hot encoder

3. CREATE Bernoulli Naïve Bayes model

4. COMBINE encoder + model into one pipeline

5. DEFINE parameter grid (alpha)

6. RUN grid search on training set using stratified CV and recall(p)

7. SELECT best estimator

8. PREDICT test set labels

9. COMPUTE accuracy, precision(p), recall(p), f1(p)

10. GENERATE confusion matrix

END

3.4.3 Pseudocode – Random Forest

START

1. USE x_train, y_train, x_test, y_test

2. CREATE one-hot encoder

3. CREATE Random Forest model

4. COMBINE encoder + model into one pipeline

5. DEFINE parameter grid (n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features)

6. RUN grid search on training set using stratified CV and recall(p)

7. SELECT best estimator

8. PREDICT test set labels

9. COMPUTE accuracy, precision(p), recall(p), f1(p)

10. GENERATE confusion matrix

END

3.5 Flowchart

Flowchart - Overall System

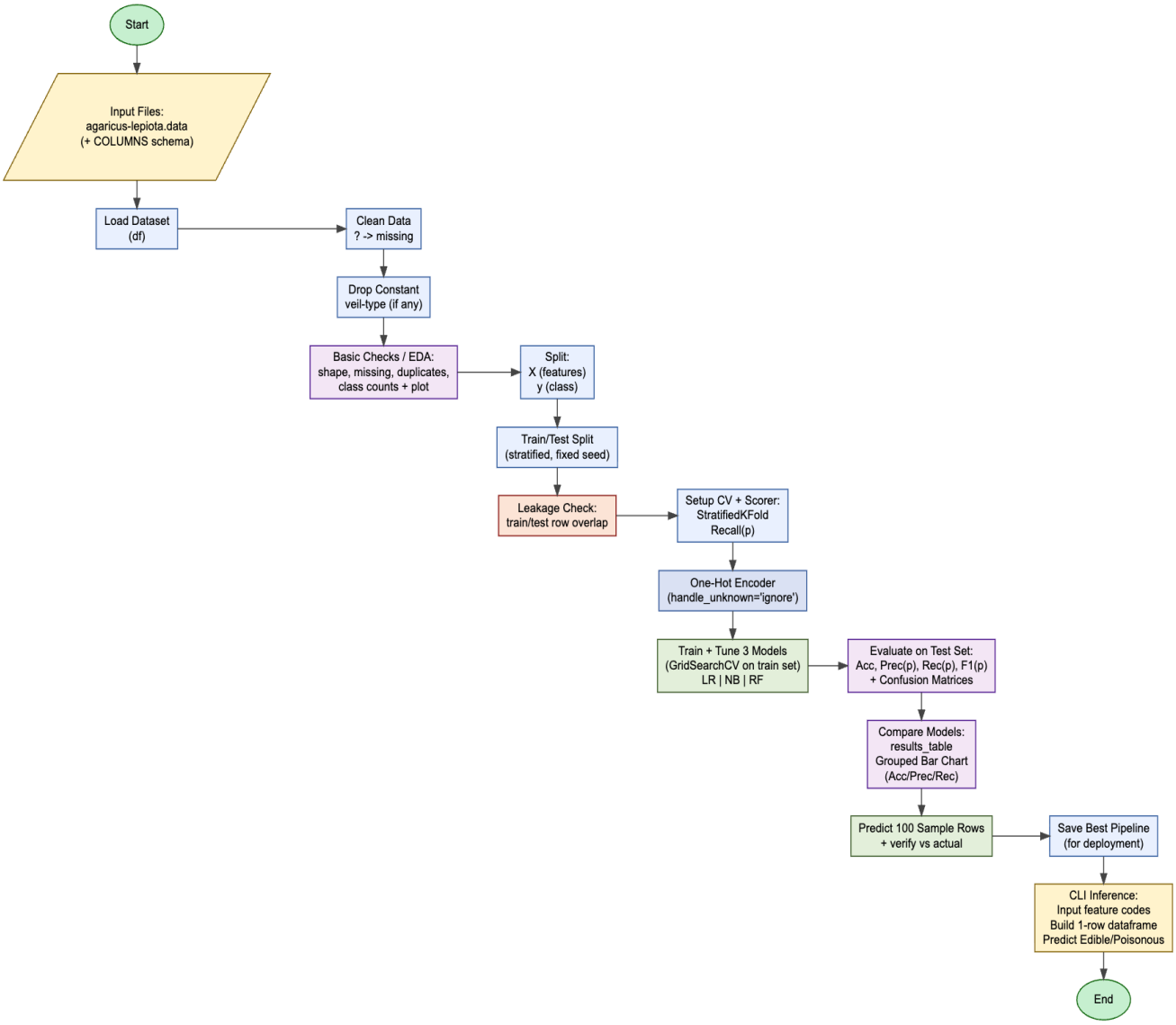

Figure 9: Flowchart of the Overall System

Flowchart - Logistic Regression

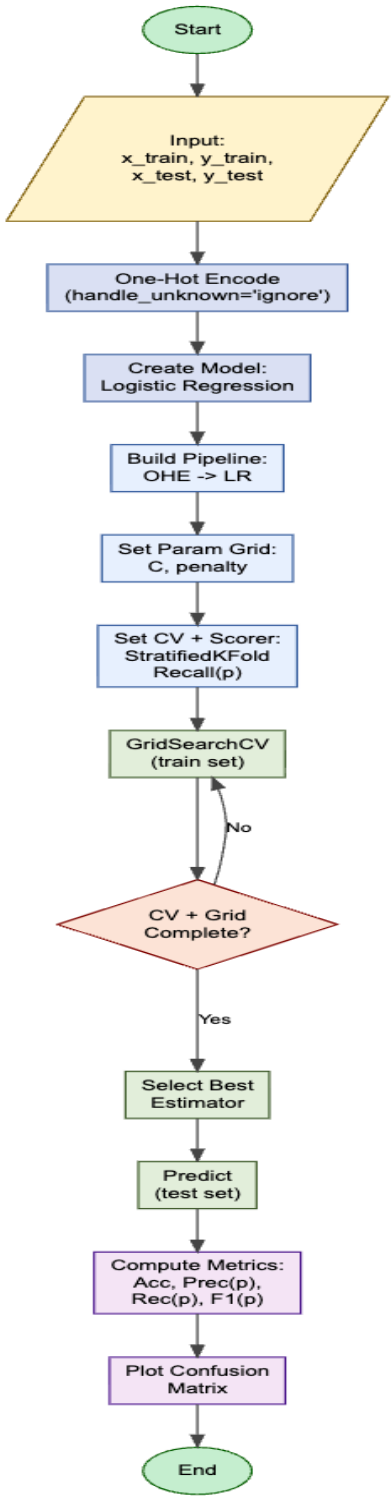

Figure 10: Flowchart for Logistic Regression

Flowchart – Naïve Bayes

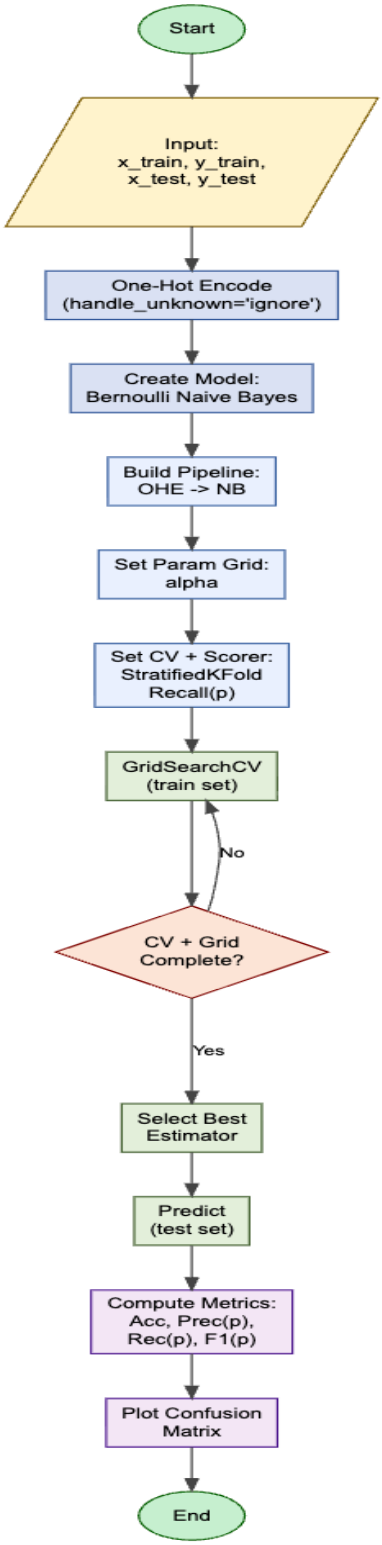

Figure 11: Flowchart for Naïve Bayes

Flowchart – Random Forest

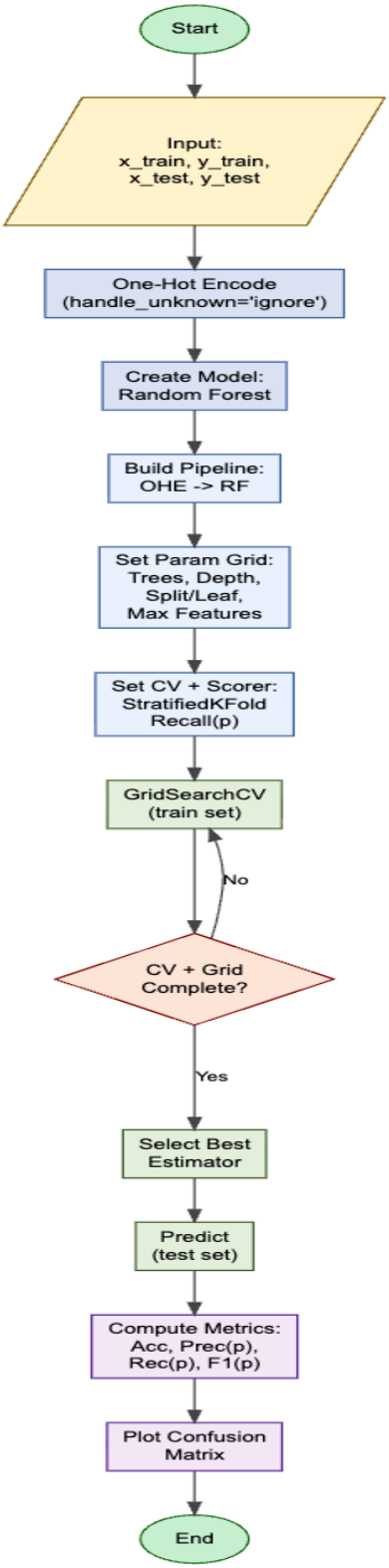

Figure 12: Flowchart for Random Forest

3.6 Data Exploration

Understanding the Dataset

Understanding of the dataset is essential because through it we can identify:

- Dataset size, structure, and datatypes.
- Missing or inconsistent values.
- Irrelevant columns.
- Preparation steps before

Observations from the dataset

- 'agaricus-lepiota.data' is the actual dataset containing one mushroom per row.
- 'agaricus-lepiota.names' is the metadata which describes what the dataset is about.
- All the attributes are categorical data (Object Datatype)
- Missing values are stored as '?', not NaN.
- Column (veil-type) has only a single unique value. As it contains no meaningful data for training models, the column can be removed.

Description of the dataset

The dataset consists of 8124 rows and 22 categorical columns which has class indicating whether a mushroom is poisonous (p)or edible (e), and the other columns explain the characteristics of the mushroom with which its edibility status is predicted.

The table describing features is in APPENDIX 1

4. Development

4.1 Development Overview

This section covers all the essential libraries required in the project, preparation of the dataset (making the dataset useful), model fitting strategy, and train-test split to ensure unbiased evaluation. After train-test split, encoding and pipeline training is done to control data leakage.

4.2 Libraries and Tools used

Python

Python was chosen as the primary development language due to the abundance of scientific computing and machine learning tools, its large and vibrant population, and usage in research in the fields of data science and high-energy physics.

Numpy

NumPy was used to process numbers with high speed and to operate with arrays effectively. It facilitates the storage of large feature matrices and label vectors and high-dimensional calculations in a compact amount of time.

Pandas

Pandas was employed in the arrangement and searching of data. It allows data loading, finding out types and structure of rows and columns, and addressing missing values during data pre-processing. Pandas is crucial for data cleaning and preparation of our project.

Matplotlib

To analyse the data and compare the results of the models Matplotlib was used to draw clear and fixed images. It was applied to plots like class balance charts, feature distribution histograms and evaluation plots like Confusion Matrix and Bar Graphs.

Scikit-learn

Scikit-learn served as the main library of the machine learning to create and test models. It was applied to train the classifiers of Logistic Regression and Random Forest and to obtain the performance measures such as Accuracy, Precision, Recall, F1-score, Confusion Matrix and other informative plots.

Importing all essential libraries

All the required libraries for the project are imported.

```
# Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, make_scorer
)

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import BernoulliNB
from sklearn.ensemble import RandomForestClassifier
```

Figure 13: Importing essential libraries.

4.3 Naming and Importing the dataset

Assigning names to categorical columns

The original dataset contains only letters as categorical values which is why it is essential to give meaningful attribute names. It is done by defining COLUMNS from dataset to ensure correct mapping.

```
# Column names from the UCI mushroom dataset
COLUMNS = [
    'class',
    'cap-shape','cap-surface','cap-color','bruises','odor',
    'gill-attachment','gill-spacing','gill-size','gill-color',
    'stalk-shape','stalk-root',
    'stalk-surface-above-ring','stalk-surface-below-ring',
    'stalk-color-above-ring','stalk-color-below-ring',
    'veil-type','veil-color','ring-number','ring-type',
    'spore-print-color','population','habitat'
]
```

Figure 14: Assigning names to columns.

Data loading

Raw dataset is loaded into a DataFrame for pre-processing by using the method pd.read_csv(). COLUMNS is passed in the method which ensures consistent structure. The loaded dataset is displayed with correct column labels.

```
#Importing the dataset
import pandas as pd
df = pd.read_csv('Data/agaricus-lepiota.data', header=None, names=COLUMNS)
df.head()
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-ring	veil-type	veil-color	ring-number	ring-type	spore-print-color	population
0	p	x	s	n	t	p	f	c	n	k	...	s	w	w	p	w	o	p	k	s
1	e	x	s	y	t	a	f	c	b	k	...	s	w	w	p	w	o	p	n	n
2	e	b	s	w	t	l	f	c	b	n	...	s	w	w	p	w	o	p	n	n
3	p	x	y	w	t	p	f	c	n	n	...	s	w	w	p	w	o	p	k	s
4	e	x	s	g	f	n	f	w	b	k	...	s	w	w	p	w	o	e	n	a

5 rows × 23 columns

Figure 15: Loading dataset into DataFrame

Shape and Information of the DataFrame

Shape and info of the dataset are shown to explain their size and basic information by using the methods df.shape() and df.info().

```
print('Shape:', df.shape)
df.info()
```

```
Shape: (8124, 23)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype 
---  ------                    --------------  ----- 
 0   class                     8124 non-null   object
 1   cap-shape                 8124 non-null   object
 2   cap-surface               8124 non-null   object
 3   cap-color                 8124 non-null   object
 4   bruises                   8124 non-null   object
 5   odor                      8124 non-null   object
 6   gill-attachment           8124 non-null   object
 7   gill-spacing              8124 non-null   object
 8   gill-size                 8124 non-null   object
 9   gill-color                8124 non-null   object
 10  stalk-shape               8124 non-null   object
 11  stalk-root                8124 non-null   object
 12  stalk-surface-above-ring  8124 non-null   object
 13  stalk-surface-below-ring  8124 non-null   object
 14  stalk-color-above-ring    8124 non-null   object
 15  stalk-color-below-ring    8124 non-null   object
 16  veil-type                 8124 non-null   object
 17  veil-color                8124 non-null   object
 18  ring-number               8124 non-null   object
 19  ring-type                 8124 non-null   object
 20  spore-print-color         8124 non-null   object
 21  population                8124 non-null   object
 22  habitat                   8124 non-null   object
dtypes: object(23)
memory usage: 1.4+ MB
```

Figure 16: Shape and info of the df

4.4 Data Preparation

Auditing the missing-marker '?'

Instead of standard missing-value formats, dataset uses '?' to represent missing/unknown values. Before cleaning the DataFrame, frequency count of '?' is done. This helps identify which attributes required missing handling and prevents errors.

```
(df == '?').sum().sort_values(ascending=False)
```

```
stalk-root                  2480
stalk-surface-above-ring       0
population                     0
spore-print-color              0
ring-type                      0
ring-number                    0
veil-color                     0
veil-type                      0
stalk-color-below-ring         0
stalk-color-above-ring         0
stalk-surface-below-ring       0
class                          0
cap-shape                      0
stalk-shape                    0
gill-color                     0
gill-size                      0
gill-spacing                   0
gill-attachment                0
odor                           0
bruises                        0
cap-color                      0
cap-surface                    0
habitat                        0
dtype: int64
```

Figure 17: Auditing missing marker '?'.

Handling Missing Values

For the dataset to be usable for modelling, '?'markers are replaced with traditional missing value (pd.NA). Missing values are then filled with the label 'missing'. This helps to preserve all rows, appropriate for categorical ML pipelines where missing values too can be taken as informative categories.

```
df_clean = df.copy()
df_clean = df_clean.replace('?', pd.NA)
df_clean = df_clean.fillna('missing')
df_clean.isnull().sum().sort_values(ascending=False)
```

```
class                       0
stalk-surface-above-ring    0
population                  0
spore-print-color           0
ring-type                   0
ring-number                 0
veil-color                  0
veil-type                   0
stalk-color-below-ring      0
stalk-color-above-ring      0
stalk-surface-below-ring    0
stalk-root                  0
cap-shape                   0
stalk-shape                 0
gill-color                  0
gill-size                   0
gill-spacing                0
gill-attachment             0
odor                        0
bruises                     0
cap-color                   0
cap-surface                 0
habitat                     0
dtype: int64
```

Figure 18: Replacing '?' with 'missing'

Null count check is performed so that it is ensured that dataset has no remaining missing values.

Checking Duplicate Records

Evaluation scores can go up due to duplicate entries. So, the cleaned dataset is checked for duplicated rows to ensure data integrity and reliable evaluation.

```
df_clean.duplicated().sum()
```

```
0
```

Figure 19: Sum of duplicate rows.

Detection and Removal of Non-informative Features

Features that have only a single unique value provide no predictive information. These features increase dimensionality without benefit when included and can complicate modelling. So, unique-category count check is done, and constant features are removed. 'Veil-type' is the constant feature in this dataset which is dropped to improve model clarity and efficiently.

```
df_clean.nunique().sort_values()
```

```
veil-type                  1
class                      2
bruises                    2
gill-attachment            2
gill-spacing               2
gill-size                  2
stalk-shape                2
ring-number                3
cap-surface                4
veil-color                 4
stalk-surface-below-ring   4
stalk-surface-above-ring   4
ring-type                  5
stalk-root                 5
cap-shape                  6
population                 6
habitat                    7
stalk-color-above-ring     9
stalk-color-below-ring     9
odor                       9
spore-print-color          9
cap-color                 10
gill-color                12
dtype: int64
```

```
# Drop constant (non-informative) column
if 'veil-type' in df_clean.columns and df_clean['veil-type'].nunique() == 1:
    df_clean = df_clean.drop(columns=['veil-type'])

print('Updated shape:', df_clean.shape)
```

```
Updated shape: (8124, 22)
```

Figure 20: Detection and Removal of Non-informative Features.

The size of column is reduced by 1 and it is now 22.

4.5 Explanatory Data Analysis

Class Distribution

Class distribution is checked to confirm the class is balanced which means similar number of edible and poisonous mushrooms in the dataset. This is necessary because accuracy alone can be misleading if a class is more dominating. So, class-sensitive metrics are later emphasised. Methods are used for value counts and to plot a bar graph. As a result, distribution of class is displayed in a numeric and visual form.

```
class_counts = df_clean['class'].value_counts()
class_counts
```

```
class
e    4208
p    3916
Name: count, dtype: int64
```

Figure 21: Code to show the class distribution in the dataset.

Visual Representation

The class distribution is balanced with almost 50/50 split of edible and poisonous mushrooms. This makes the evaluation metrics more reliable, accuracy more meaningful and stratified splitting works better.

```
import matplotlib.pyplot as plt

plt.figure()
plt.bar(class_counts.index, class_counts.values)
plt.title('Class Distribution (Target: class)')
plt.xlabel('Class')
plt.ylabel('Count')
plt.show()
```

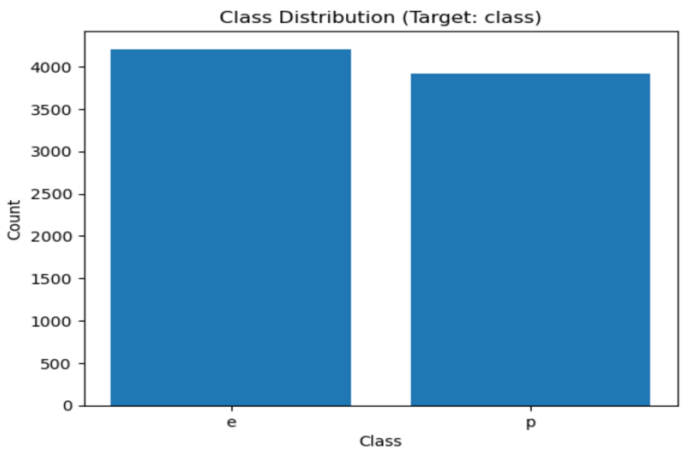

Figure 22: Bar Graph to display Class Distribution.

4.6 Train-Test Split

Feature-Target Separation and Stratified Split

The dataset contains features and target, and they should be separated to prevent leakage. If target 'y' mistakenly stays inside feature 'X', the model can become faulty and produce unrealistically high accuracy.

The dataset is divided into training set which is used to train and tune the model and test set which is kept aside and used only at the end to evaluate the performance. 80% of the data is separated for training and 20% for testing using stratify = y to preserve original class proportions in both subsets. Test set was not used during training or hyperparameter tuning to ensure the final evaluation provides valid estimation of the unseen data.

```
from sklearn.model_selection import train_test_split

X = df_clean.drop(columns=['class'])
y = df_clean['class']

# Target leakage check: 'class' must not be in X
print('class' in X.columns)

x_train, x_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

x_train.shape, x_test.shape, y_train.shape, y_test.shape
```

```
False
((6499, 21), (1625, 21), (6499,), (1625,))
```

Figure 23: Feature/Target separatioin and stratified train-test split.

As a result, dataset is separated into train set and test set which contains 6499 and 1625 rows respectively.

Duplicate Overlap Check

Overlap check is done to confirm that identical feature rows do not appear in both splits. It prevents inflated performance due to the same samples appearing in both sets. This results into the increased credibility of evaluated results.

```
# Check exact row overlap between train and test (feature rows)
train_rows = set(map(tuple, x_train.values))
test_rows = set(map(tuple, x_test.values))
overlap = len(train_rows.intersection(test_rows))
overlap
```

```
0
```

Figure 24: Checking if duplicates exist in train-test sets.

4.7 Model Fitting Strategy

Cross-Validation and Scoring Metric Selection

Hyperparameters are tuned by using a stratified 5-fold cross-validation technique in a reliable way. The primary scoring metric during tuning is **Recall** for poisonous class (pos_label='p'). Predicting a poisonous mushroom as edible is the most critical error which is why model selection prioritises detecting poisonous cases; it is a safety-oriented decision.

```
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import make_scorer, recall_score
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Recall for poisonous as the positive class
recall_p = make_scorer(recall_score, pos_label='p')
cv, recall_p
```

```
(StratifiedKFold(n_splits=5, random_state=42, shuffle=True),
 make_scorer(recall_score, response_method='predict', pos_label=p))
```

Figure 25: Cross-Validation and Scoring Metric Selection

One-Hot Encoding inside the Pipeline

All the characteristics in this data are categorical. Categories thus must be one-hot encoded to become numeric signs to the ML algorithms. Stated more importantly, the encoder is incorporated in a Pipeline such that the encoder is only fitted using training data with each fold of cross-validation and then the encoder is used with validation/test data. This helps avoid leakage and realistic evaluation.

The encoder takes handle_unknown= ignore to be stable when a category may occur in a test data set and was not included in training data. Sparse_output=False is used to be compatible with the chosen estimators.

```
from sklearn.preprocessing import OneHotEncoder
# One-Hot Encoder for categorical variables
# sparse_output=False ensures compatibility with all models (including Random Forest)
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
```

Figure 26: Leakage-proof preprocessing using one-hot encoding.

4.8 Logistic Regression

Logistic Regression is a supervised classification algorithm. It learns a set of weights of the input features and create a probability score that a sample is of a particular class (e.g., poisonous). A strong linear baseline classifier with regularisation is employed i.e. Logistic Regression. The Pipeline is developed based on two steps, namely, one-hot encoding and Logistic Regression. The grid of regularisation strength (C) and types of penalty (l1, l2) used as hyperparameters to train a model is small and of coursework-scale size. Cross-validated Recall(p) is the motivation of selection. The purpose of using Logistic Regression as a baseline model in this project is that it is easy, quick to train and can be used as a reference point to other more complicated methods.

Fitting Logistic Regression Model

Fitting is executed by the method lr_grid.fit (x_train, y_train). GridSearchCV fits multiple pipeline instances across stratified folds and best performing pipeline is refitted on the training set.

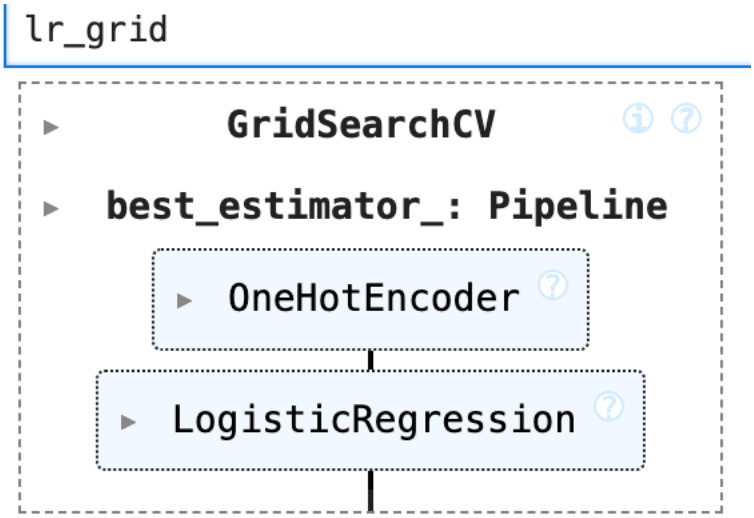

Figure 27: Fitting Logistic Regression Model using GridSearchCV.

Logistic Regression pipeline and hyperparameter tuning using GridSearchCV

Complete training pipeline is built for Logistic Regression. Pipeline is imported to one-hot encode all categorical features and then train Logistic Regression. GridSearchCV is used to select the best hyperparameter automatically in a leakage-safe way that maximises poisonous-class recall. It tests multiple values of C and penalty using Cross-Validation. The best pipeline and its CV score are returned for final testing.

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

lr_pipe = Pipeline([
    ('ohe', ohe),
    ('model', LogisticRegression(solver='liblinear', max_iter=2000, random_state=42))
])

lr_param_grid = {
    'model__C': [0.1, 1, 10],
    'model__penalty': ['l1', 'l2']
}

lr_grid = GridSearchCV(
    lr_pipe,
    lr_param_grid,
    scoring=recall_p,
    cv=cv,
    n_jobs=-1
)

lr_grid.fit(x_train, y_train)
lr_grid.best_params_, lr_grid.best_score_
```

```
({'model__C': 10, 'model__penalty': 'l1'}, 0.9993620414673046)
```

Figure 28: Logistic Regression pipeline and hyperparameter tuning using GridSearchCV

Outcomes:

Performance Metrics

The best pipeline of Logistic Regression (lr_best = lr_grid.best_estimator_) was tested on the unseen hold-out test set after tuning. lr_best.predict(x_test) was used to generate prediction and Accuracy, Precision(p), Recall(p) and F1(p) were used to measure performance. Accuracy is a measure of the total correctness of both classes whereas Precision(p) and Recall(p) are measures that specifically focus on the poisonous class. Recall(p) is highlighted since it is used to measure the proportion of poisonous mushrooms actually detected that were actually positive, and it was this that directly correlates to the safety of the model.

The Logistic Regression model in the reported results also scored perfectly on the test-set (Accuracy = 1.0, Precision(p) = 1.0, Recall(p) = 1.0, F1(p) = 1.0). It shows that, with the selected split and evaluation scheme, the tuned model has taken the correct decision on all edible and poisonous mushrooms in the test set, with no false positives and no false negatives on the classification of poisonous mushrooms.

```
from sklearn.metrics import accuracy_score, precision_score, f1_score

# Evaluate best Logistic Regression on hold-out test set
lr_best = lr_grid.best_estimator_
y_pred_lr = lr_best.predict(x_test)

acc_lr  = accuracy_score(y_test, y_pred_lr)
prec_lr = precision_score(y_test, y_pred_lr, pos_label='p')
rec_lr  = recall_score(y_test, y_pred_lr, pos_label='p')
f1_lr   = f1_score(y_test, y_pred_lr, pos_label='p')

acc_lr, prec_lr, rec_lr, f1_lr
```

```
(1.0, 1.0, 1.0, 1.0)
```

Figure 29: Logistic Regression performance metrics (Accuracy, Precision(p), Recall(p), F1(p)).

Confusion Matrix Interpretation

According to the confusion matrix, there are none of the misclassifications in the hold-out test set of the Logistic Regression. 842 edible mushrooms were correctly identified to be edible and 783 poisonous mushrooms were correctly identified to be poisonous. No edible samples were wrongly labelled as poisonous and no samples that were poisonous were wrongly labelled as edible. Interestingly, the fact that the prediction of the poisonous mushrooms as edible was not achieved proves that the safety performance in question performed as best as possible given the split considered.

```
# Confusion matrix plot (Logistic Regression)

from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred_lr,
    labels=["e", "p"],
    display_labels=["e", "p"],
    cmap="Blues"
)
plt.title("Confusion Matrix — Logistic Regression (Test Set)")
plt.show()
```

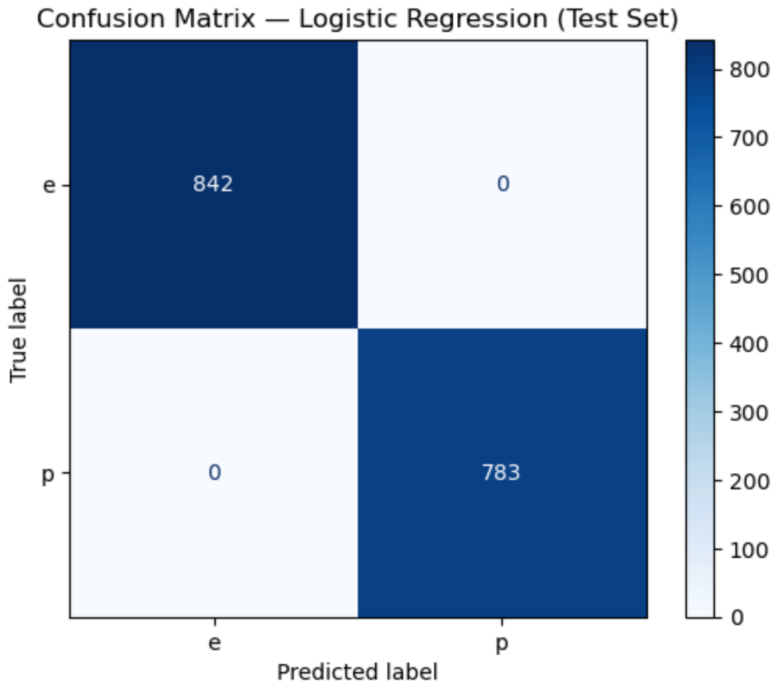

Figure 30: Confusion Matrix for Logistic Regression

Key Observations:

Accuracy

The Logistic Regression model had an accuracy of 1.0 implying that 100%. owing to the hold-out test set, the model accurately identified the mushrooms. This shows overall total correctness in the assessed split and pipeline set-up.

Precision(p)

Precision of the poisonous class, Precision(p) = 1.0, represents the fact that all those mushrooms that were forecasted as poisonous are, in fact, poisonous. This can be supported by the confusion matrix, which displays the 0 edible mushrooms as wrongly identified as poisonous. Maximum precision is a good quality since it lowers the false alarms and shows that the toxic predictions are sound.

Recall(p)

Recall of the poisonous class, Recall(p) = 1.0, is that all mushrooms which were really poisonous in the test set were detected by the model. Notably, the confusion matrix demonstrates that zero poisonous mushrooms are classified as edible instead of being zero, and it removes the most dangerous type of error in this application. The outcome is the best the safety performance that can be achieved according to the split that is being considered.

F1(p)

The F1 score of the poisonous category, F1(p) = 1.0, indicates an ideal combination of Precision(p) and Recall(p). Being as high as precision and recall, the F1 score proves that the model is both efficient at identifying the presence of poisonous mushrooms and resists false positive prediction of poisonous mushrooms.

Safety Interpretation

The worst failure mode in terms of safety is taking into account a poisonous mushroom as edible food. The custom confusion mat of the Logistic Regression proves that the test set did not possess such a mistake. Hence, in the considered protocol, the Logistic Regression offers the most effective performance related to safety consideration.

4.9 Naïve Bayes

Naive bayes is a probabilistic classifier which operates under the Bayes theorem. It uses the estimates of the probability of observed values of features to each class and determines the most probable class. Bernoulli Naive Bayes is appropriate when One-hot encoding is used since it can be presented as binary indicator features. The model is put in place with the same pipeline structure to maintain leakage safety. GridSearchCV is again used to tune the smoothing parameter alpha with an approximation being selected one again that maximizes cross-validated Recall(p). It has been made part of this report since it is a common foundation on which the classification activity since it can be effective with categorical data that are structured and is computationally economical and offers a handy comparison to Logistic Regression and Random Forest.

Fitting Naïve Bayes Model

The method nb_grid.fit(x_train, y_train) is used to fit the Naïve Bayes Model. GridSearchCV trains several instances of pipelines each on stratified folds and identifies the best-performing pipeline based on Recall(p). The optimal pipeline is then automatically refitted on the entire training data (refit=True default) so that nb_grid.best_estimator is a trained model that is ready to be evaluated on the hold-out test data set.

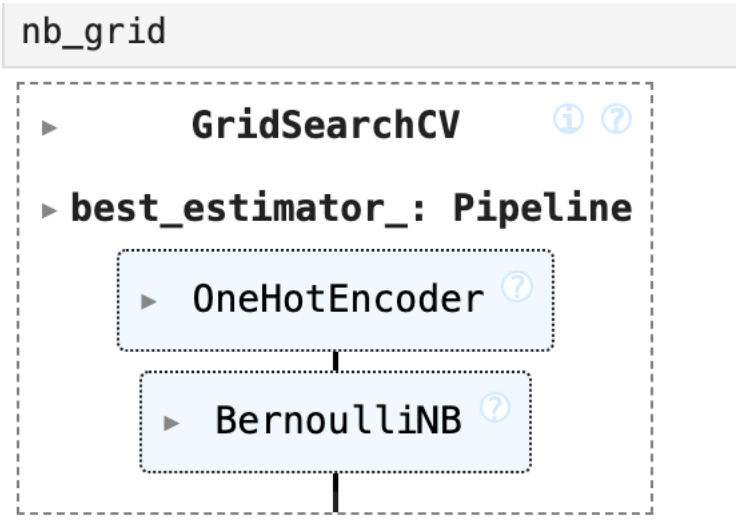

Figure 31: Fitting Naïve Bayes Model using GridSearchCV.

Naive Bayes pipeline and hyperparameter tuning using GridSearchCV

An entire Naive Bayes training pipeline is constructed by using OneHotEncoder (to transform categorical mushroom features into binary features) followed by application of BernoulliNB which can be trained with one-hot encoded inputs. Then the GridSearchCV optimizes the main smoothing parameter alpha of the model by the Stratified K-Fold cross-validation and scoring criterion (poisonous-class recall) of the project. The search determines each alpha value, picks the highest-performing Naive Bayes pipeline, and returns its optimal parameterization and cross-validated score which are both used to evaluate the final test-set performance.

```
from sklearn.naive_bayes import BernoulliNB

nb_pipe = Pipeline([
    ('ohe', ohe),
    ('model', BernoulliNB())
])

nb_param_grid = {
    'model__alpha': [0.1, 0.5, 1.0, 2.0]
}

nb_grid = GridSearchCV(
    nb_pipe,
    nb_param_grid,
    scoring=recall_p,
    cv=cv,
    n_jobs=-1
)

nb_grid.fit(x_train, y_train)
nb_grid.best_params_, nb_grid.best_score_
```

```
({'model__alpha': 0.1}, 0.9179795262189747)
```

Figure 32: Naive Bayes pipeline and hyperparameter tuning using GridSearchCV.

Outcomes:

Performance Metrics

Unseen hold-out test set after tuning to optimize Naïve Bayes on the unseen hold-out test set The tuning of Naive Bayes on the unseen hold-out test set was assessed by the best pipeline using nb_best = nb_grid.best-estimator. To measure the performance, predictions were made based on nb_best.predict(x_test). Accuracy, Precision(p), Recall(p) and F1(p) were calculated to measure the performance. Accuracy displays general correctness in performance on the two classes and Precision(p), Recall(p), and F1(p) are specifically targeted at poisonous classification. The Recall(p) is of special interest in this project since it will be used to estimate the percentage of poisonous mushrooms that are correctly recognized, which is directly proportional to the safety of the classifier.

The results of the Naive Bayes model were the following on the test-set:

Accuracy = 0.9582, Precision(p) = 0.9877, Recall(p) = 0.9246, F1(p) = 0.9551.

These results illustrate that, these are good overall performances with very high accuracy of the poisonous predictions and that there would be low recall compared to Logistic Regression. It implies that even though the model is highly dependable in the case of predicting whether a mushroom is poisonous, it nevertheless does not recognize a group of mushrooms that are poisonous (which is the most important type of error in this field).

```
# Evaluate best Naïve Bayes on hold-out test set
nb_best = nb_grid.best_estimator_
y_pred_nb = nb_best.predict(x_test)

acc_nb  = accuracy_score(y_test, y_pred_nb)
prec_nb = precision_score(y_test, y_pred_nb, pos_label='p')
rec_nb  = recall_score(y_test, y_pred_nb, pos_label='p')
f1_nb   = f1_score(y_test, y_pred_nb, pos_label='p')

acc_nb, prec_nb, rec_nb, f1_nb
```

```
(0.9581538461538461,
 0.9877216916780355,
 0.9246487867177522,
 0.9551451187335093)
```

Figure 33: Naive Bayes Performance Metrics.

Confusion Matrix Interpretation

The confusion matrix allows having the full breakdown of the correct and incorrect predictions by the stages and the two classes. 833 edible mushrooms were recognized to be edible correctly. There were 9 edible mushrooms that were predicted as poisonous. The number of mushrooms that were identified as poisonous correctly was 724. There were 59 mushrooms that were mistaken as edible mushrooms. The most severe outcome is that 59 mushrooms were predicted to be edible, and this is false negatives of the poisonous group. These faults decrease Recall(p) and are safety concerns.

```
# Confusion matrix plot (Naïve Bayes)

from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred_nb,
    labels=["e", "p"],
    display_labels=["e", "p"],
    cmap="Blues"
)
plt.title("Confusion Matrix — Naïve Bayes (Test Set)")
plt.show()
```

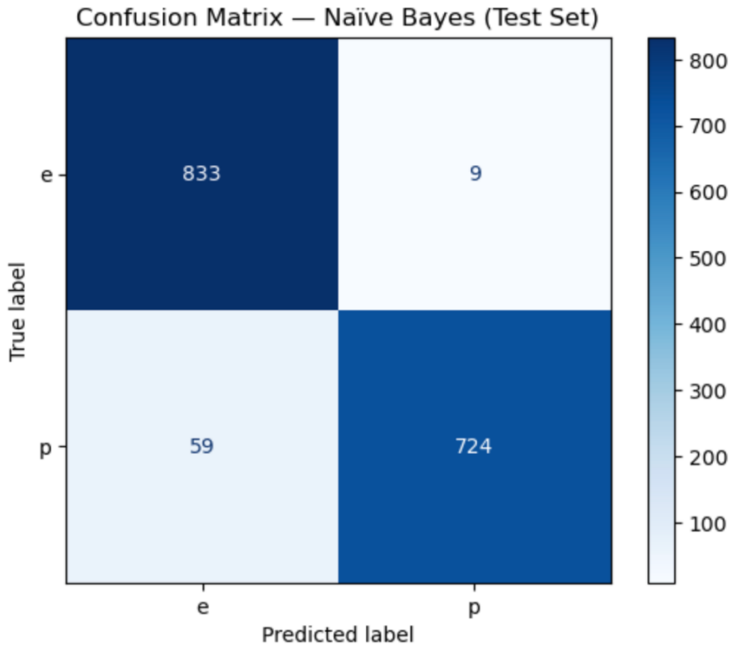

Figure 34: Confusion Matrix for Naive Bayes.

Key Observations:

Accuracy

The accuracy of 0.9582 tell that about 95.8 percent of all test mushrooms were overall classified correctly. Nonetheless, accuracy is not enough in safety-critical classification as it fails to differentiate between non harmful errors and errors which are risky.

Precision(p)

The precision(p) of 0.9877 suggests that whenever the model predicts that a mushroom is poisonous, it is right on 98.8. This is in accordance with the confusion matrix that demonstrates only 9 edible mushrooms were labeled as those that are poisonous. Naive Bayes generates minimal false alarms to poisonous classification in real sense.

Recall(p)

The recall(p) of 0.9246 implies that the model is identifying a true moulded mushroom that is truly poisonous correctly about 92.5% of the time. A confusion matrix indicates that 59 poisonous mushrooms were classified as edible, and this directly decreases Recall(p). As the most safety-critical failure mode is poisonous-to-edible misclassification, this is the main deficiency of Naive Bayes when compared to the more powerful-performers.

F1(p)

F1(p) 0.9551 represents the trade-off between Precision(p) and Recall(p). It is a high score on the balance, with a major constraint being the lower Recall(p). This implies that Naive Bayes excels over false positive avoidance but can still excel in general because it does not have on some poisonous detection sensitivity as compared to the Logistic Regression.

Safety Interpretation

The major risk associated with a safety viewpoint is the presence of mushrooms that are poisonous that are anticipated to be edible. Although the performance is good, the Naive Bayes confusion matrix validates the non-trivial number of false negatives on the class of poisonous, thus, is not as effective as a model, which supports the Recall(p) with higher numbers.

4.10 Random Forest

Random Forest is an ensemble machine learning procedure, which creates a variety of decision trees and integrates the results of these trees to provide a resulting prediction. Random Forest is also applicable to model non-linear relationships and interaction between features which may not be pre-woven into linear models. The combining of a Pipeline (one-hot encoder → RandomForestClassifier) is built and the important parameters during tree-growth and ensemble are optimized through GridSearchCV. The search space can consist of the number of trees, maximum depth, split and leaf threshold, and feature sampling strategy. Selection Like other models, it is selected based on cross-validated Recall(p).

Fitting Random Forest Model

Model is fit using the method rf_grid.fit(x_train, y_train). The gridSearchCV fits a series of pipelines on stratified cross-validation folds, and the configuration with the highest values of Recall(p) may be chosen (with the poisonous class (p) being the positive one). The most successful pipeline is refitted automatically on the entire training (refit=True by default), such that rf_grid.bestestimator is a fully trained model at the ready to be tested on unseen test data.

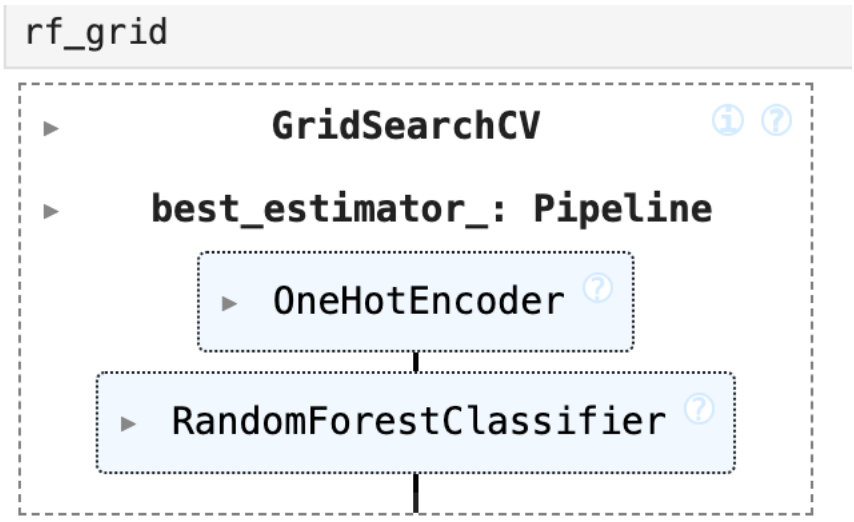

Figure 35: Fitting Random Forest Model.

Random Forest pipeline and hyperparameter tuning using GridSearchCV

OneHotEncoder and RandomForestClassifier are used to build a full pipeline for the training of Random Forest. Then GridSearchCV is utilized to optimize important hyperparameters of the Random Forest which are: the number of trees (n_estimators), the tree depth (max_depth), split controls (min_samples_split, min_samples_leaf), and the feature sampling strategy (max_features). The combination of the parameters is cross-validated by Stratified K-Fold on the training set and optimizes the project safety-oriented measure (recall by the poisonous class, p). The most successful pipeline of Random Forest and its cross-validation score are sent back to undergo final evaluation on the test-set.

```
from sklearn.ensemble import RandomForestClassifier

rf_pipe = Pipeline([
    ('ohe', ohe),
    ('model', RandomForestClassifier(random_state=42))
])

rf_param_grid = {
    'model__n_estimators': [100, 300, 500],
    'model__max_depth': [None, 10, 20],
    'model__min_samples_split': [2, 5],
    'model__min_samples_leaf': [1, 2],
    'model__max_features': ['sqrt', 'log2']
}

rf_grid = GridSearchCV(
    rf_pipe,
    rf_param_grid,
    scoring=recall_p,
    cv=cv,
    n_jobs=-1
)

rf_grid.fit(x_train, y_train)
rf_grid.best_params_, rf_grid.best_score_
```

```
({'model__max_depth': None,
  'model__max_features': 'sqrt',
  'model__min_samples_leaf': 1,
  'model__min_samples_split': 2,
  'model__n_estimators': 100},
 1.0)
```

Figure 36: Random Forest pipeline and hyperparameter tuning using GridSearchCV.

Outcomes:

Performance Metrices

Tuning was done to assess the best random forest pipeline which is rf_grid.best_estimator on the test set that has not been observed (hold-out test set). rf_best.predict(x_test) was used to give predictions and Accuracy, Precision(p), Recall(p), and F1(p) were calculated to estimate performance. The summary of accuracy reflects the overall correctness between the two categories, whereas Precision(p) and Recall(p) concentrate on the poisonous category, which is the target of safety in this project. The focus on Recall(p) is caused by the fact that the false negative (poisonous being predicted as edible) is the worst type of error.

Random Forest model was observed to perform perfectly when tested on a set of random cases: Accuracy = 1.0, Precision(p) = 1.0, Recall(p) = 1.0, F1(p) = 1.0.

This suggests that with the split and pipeline-based training protocol under consideration, the tuned Random Forest would label all test mushrooms as edible and no mushrooms as poisonous with no false positives and false negatives described.

```
# Evaluate best Random Forest on hold-out test set
rf_best = rf_grid.best_estimator_
y_pred_rf = rf_best.predict(x_test)

acc_rf  = accuracy_score(y_test, y_pred_rf)
prec_rf = precision_score(y_test, y_pred_rf, pos_label='p')
rec_rf  = recall_score(y_test, y_pred_rf, pos_label='p')
f1_rf   = f1_score(y_test, y_pred_rf, pos_label='p')

acc_rf, prec_rf, rec_rf, f1_rf
```

```
(1.0, 1.0, 1.0, 1.0)
```

Figure 37: Random Forest Performance Metrics.

Confusion Matrix Interpretation

The confusion matrix gives an in-depth analysis of both correct and incorrect predictions. In the case of the Random Forest classifier, the confusion matrix is: Eight hundred and forty two edible mushrooms were rightfully identified as edible. There were 0 edible mushrooms that were labeled as poisonous. 783 mushrooms that were poisonous were identified as such. 0 harmful mushrooms were labelled as edible. The fact that the absence of poisonous mushrooms was the predicted edible gives zero false negatives based on safety criteria in the categorization of poisonous. Recall(p) = 1.0 implies maximum safety performance under the considered set.

```
# Confusion matrix plot (Random Forest)

from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred_rf,
    labels=["e", "p"],
    display_labels=["e", "p"],
    cmap="Blues"
)
plt.title("Confusion Matrix — Random Forest (Test Set)")
plt.show()
```

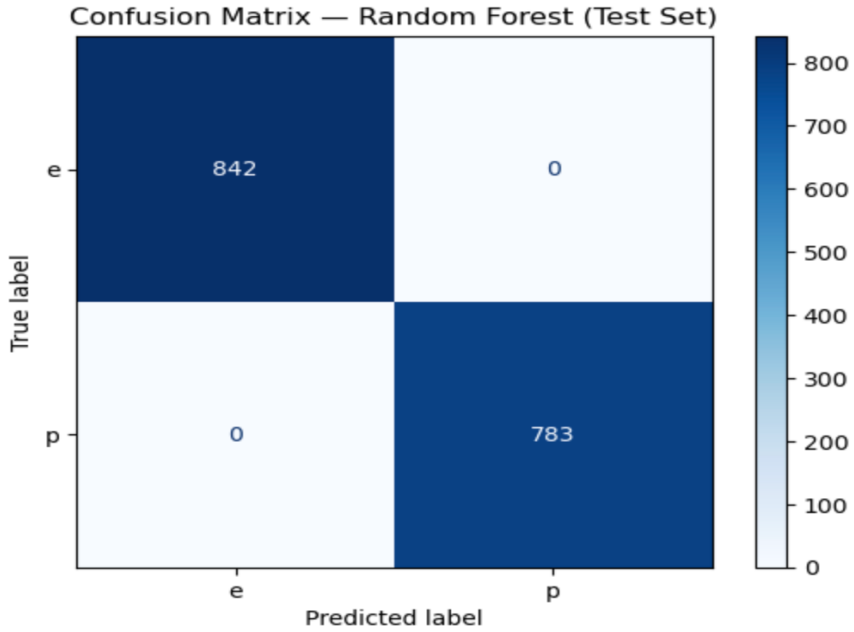

Figure 38: Confusion Matrix for Random Forest.

Key Observations:

Accuracy

The value of 1.0 of Accuracy shows that the sample of the data was all classified by Random Forest as correct, so the overall correctness is flawless against the evaluation protocol.

Precision(p)

Precision(p) of 1.0 means that all mushrooms that were predicted to be poisonous had been actually poisonous. This is supported by the confusion matrix because there were no edible mushrooms occurrences that were incorrectly identified as poisonous, i.e., the model did not generate any false alarms on the class of poisonous mushrooms.

Recall(p)

The Recallp1.0 of 1.0 means that the model has been able to identify all mushrooms that were really poisonous in the test set. The confusion matrix confirms the fact that there were zero types of poisonous mushrooms that were mistaken as edible ones to get rid of the type of error that could be the most critical in the context of safety matters.

F1(p)

The value of F1(p) The perfect management between Precision(p) and Recall(p) is 1.0. The F1 score shows that the original metrics are both maximal and therefore, proves the fact that the Random Forest offers high poisonous detection and does not reduce reliability.

Safety Interpretation

In terms of safety the poorest mode of failure is a prediction of a poisonous mushroom as edible. Random Forest did not commit any errors in the test set which means the maximum performance in terms of safety with the split being assessed.

Advantages and Disadvantages of the Models Used

Table 1: Table for Advantages and Disadvantages of all 3 algorithms.

Model	Advantages	Disadvantages
Logistic Regression	1) Easy to interpret. 2) Fast to train and predict, it also works well for one hot encoded data.	1) If used for complex patterns, it can struggle 2)Needs tuning to avoid underfitting/overfitting.
Bernoulli Naïve Bayes	1) It is efficient in binary one hot encoded data. 2) Performs well on categorical data.	1) Assumes that features are independent. 2) There is drop in performance when features are highly dependent.
Random Forest	1) It is generally very accurate. 2) Handles non-linear patterns very well.	1) It is harder to interpret and explain. 2) It is slower and heavier than the other two algorithms.

5. Comparisons

Test Set Performance Table

Performance of 3 models using unseen test set results is compared. A table is created containing every model's:

- CV Best Recall: Best CV Recall for poisonous class achieved during training.
- Test Accuracy: Overall accuracy of the model on unseen test data.
- Test Precision: Detection rate of poisonous mushrooms.
- Test F1: Mean of Precision and Recall.

Table 2: Comparision table of Models based on Performance Metrics.

```
results_table = pd.DataFrame({
    'Model': ['Logistic Regression', 'Naïve Bayes', 'Random Forest'],
    'CV Best Recall(p)': [lr_grid.best_score_, nb_grid.best_score_, rf_grid.best_score_],
    'Test Accuracy': [acc_lr, acc_nb, acc_rf],
    'Test Precision(p)': [prec_lr, prec_nb, prec_rf],
    'Test Recall(p)': [rec_lr, rec_nb, rec_rf],
    'Test F1(p)': [f1_lr, f1_nb, f1_rf]
})
results_table
```

	Model	CV Best Recall(p)	Test Accuracy	Test Precision(p)	Test Recall(p)	Test F1(p)
0	Logistic Regression	0.999362	1.000000	1.000000	1.000000	1.000000
1	Naïve Bayes	0.917980	0.958154	0.987722	0.924649	0.955145
2	Random Forest	1.000000	1.000000	1.000000	1.000000	1.000000

Bar Graph for Comparison of Test Metrics

A bar chart is created based on the test-set values of Accuracy, Precision(p) and Recall(p) to enhance the understanding of the results. This graphical comparison shows that the performance of Logistic Regression and Random Forest is consistently optimum, in all the three aspects. Naive Bayes demonstrates a significant amount of precision and a visibly lower recall as an example that it has higher chances of overlooking poisonous mushrooms compared to the other two models.

The given bar chart is useful especially since it displays the trade-off profile of any one model. Naive Bayes is very precise in general, but the lower Recall(p) value implies lower safety profile than the models with perfect poisonous detectability.

```
import matplotlib.pyplot as plt

results_table.set_index("Model")[["Test Accuracy", "Test Precision(p)", "Test Recall(p)"]].plot(
    kind="bar",
    figsize=(8,4)
)

plt.title("Test Metrics by Model")
plt.ylabel("Score")
plt.xticks(rotation=15)
plt.legend(loc="upper left", bbox_to_anchor=(1.02, 1))
plt.tight_layout()
plt.show()
```

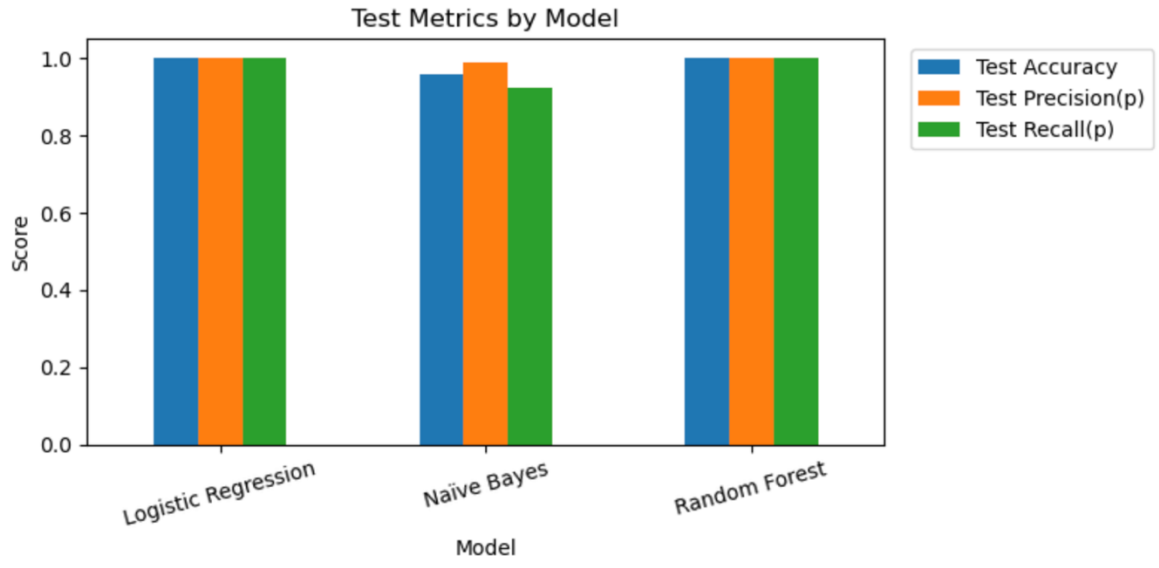

Figure 39: Bar Graph displaying comparison of model's performance.

Final Interpretation and Model Selection

Recall(p) is chosen as the preferred method of model selection since the worst possible mistake been made is picking up poisonous mushrooms and declaring them edible. Within this criterion, Random Forest and Logistic Regression offer the best safety results on the hold-out test dataset including zero poisonous to edible misclassifications. Naive Bayes though effective and accurate, results in an estimate of safety-critical false negatives to the poisonous group.

Considering the outcome of this assessment plan, the most appropriate models to implement during the prediction phase will be the Logistic Regression and the Random Forest especially where the main need is safety-based detection of the poisonous mushrooms.

6. Evaluation

Prediction on Unseen Dataset

Prediction is done using the best model (Random Forest). It verifies that the trained model can produce accurate predictions. 100 random samples are selected. Then, y labels are predicted. The predicted y labels are compared with true y labels. A table is then displayed showing actual vs predicted. Correct predictions are counted, accuracy, precision and recall are computed and a confusion matrix is plotted.

```
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# True labels for the same 100 rows you predicted
y_true_100 = df_clean.loc[X_100.index, "class"]          # 'e' or 'p'
y_pred_100 = best_model.predict(X_100)                   # 'e' or 'p'

# Table: actual vs predicted
check_100 = pd.DataFrame({
    "Actual": y_true_100.map({"e": "Edible", "p": "Poisonous"}),
    "Predicted": pd.Series(y_pred_100, index=X_100.index).map({"e": "Edible", "p": "Poisonous"})
})
check_100["Correct?"] = (check_100["Actual"] == check_100["Predicted"])

display(check_100.head(20))
print("Correct predictions:", check_100["Correct?"].sum(), "/ 100")

# Metrics (treat poisonous as the positive class)
acc = accuracy_score(y_true_100, y_pred_100)
prec = precision_score(y_true_100, y_pred_100, pos_label="p")
rec = recall_score(y_true_100, y_pred_100, pos_label="p")

print(f"Accuracy:  {acc:.4f}")
print(f"Precision (poisonous): {prec:.4f}")
print(f"Recall (poisonous):    {rec:.4f}")
```

Figure 40: Code to show table of actual vs predicted values result.

Table 3: Table showing Actual Class vs Predicted Class.

	Actual	Predicted	Correct?
1971	Edible	Edible	True
6654	Poisonous	Poisonous	True
5606	Poisonous	Poisonous	True
3332	Edible	Edible	True
6988	Poisonous	Poisonous	True
5761	Poisonous	Poisonous	True
5798	Poisonous	Poisonous	True
3064	Poisonous	Poisonous	True
1811	Edible	Edible	True
3422	Edible	Edible	True
3555	Edible	Edible	True
5317	Poisonous	Poisonous	True
706	Edible	Edible	True
1075	Edible	Edible	True
233	Edible	Edible	True
3480	Edible	Edible	True
1557	Edible	Edible	True
7634	Poisonous	Poisonous	True
828	Edible	Edible	True
2534	Edible	Edible	True

```
Correct predictions: 100 / 100
Accuracy:  1.0000
Precision (poisonous): 1.0000
Recall (poisonous):    1.0000
```

***Confusion* Matrix for the Predicted Results**

```
# Confusion matrix for the 100 rows

from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_true_100, y_pred_100,
    labels=["e", "p"],
    display_labels=["Edible", "Poisonous"],
    cmap="Blues"
)
plt.title("Confusion Matrix — 100 Sample Predictions")
plt.show()
```

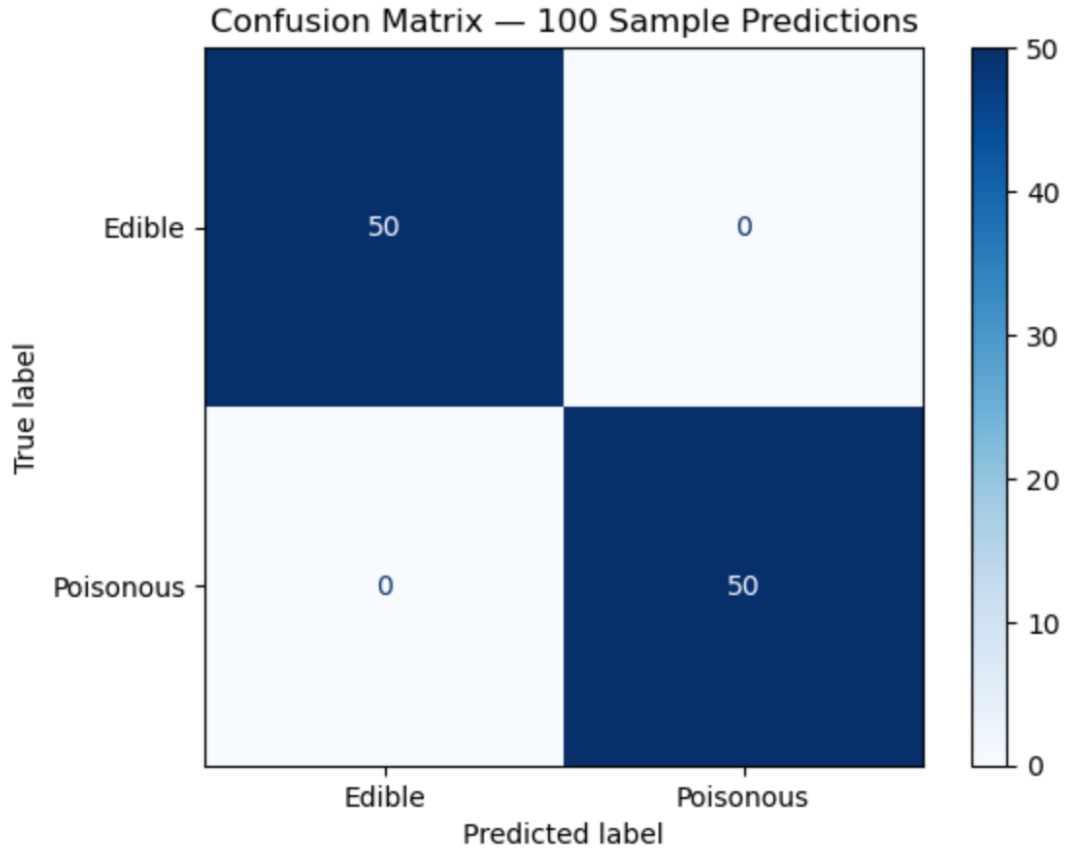

Figure 41: Confusion Matrix for Predicted Results.

CLI Prototype

First, model is exported to be used in python file.

```
# Model export for CLI usage
import joblib

X = df_clean.drop(columns=["class"])
y = df_clean["class"]

best_model = rf_grid.best_estimator_
best_model.fit(X, y)

joblib.dump(best_model, "mushroom_model.joblib")
print("Saved mushroom_model.joblib")
```

```
Saved mushroom_model.joblib
```

Figure 42: Exporting the best model for CLI usage.

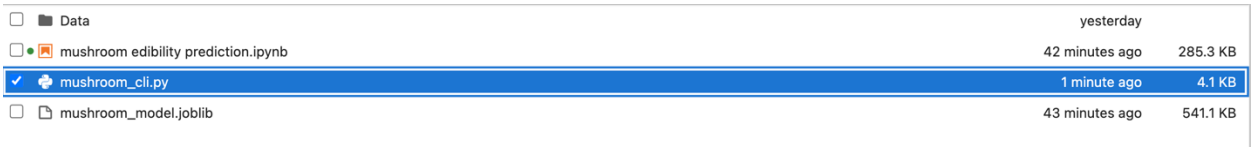

Figure 43: Project Folder Structure

```
def main() -> None:
    """
    Run a single prediction from terminal input.

    Workflow:
    1) Load the trained model pipeline from disk.
    2) Read one comma-separated row from the user (22 feature values).
    3) Clean input values to match notebook cleaning rules (e.g., '?' -> 'missing').
    4) Build a one-row DataFrame with the exact feature columns expected by the model.
    5) Predict edible/poisonous and display results.
    """

    # 1) Load the trained model pipeline
    # The saved object is a scikit-learn Pipeline (e.g., OneHotEncoder + model),
    # trained in the notebook and exported using joblib.
    model = joblib.load("mushroom_model.joblib")

    # 2) Determine the exact input features expected by the trained model
    # This prevents feature mismatch errors (e.g., if a constant column was dropped in training).
    features = list(model.feature_names_in_)

    # 3) Prompt the user for one row of input (22 feature codes only)
    print("\nMushroom Edibility Predictor")
    print("Paste 22 comma-separated feature codes (NO class label).")
    print("Order (dataset features, excluding class):")
    print(", ".join(COLUMNS[1:]))
    print("\nExample (22 values):")
    print("x,s,n,t,p,f,c,n,k,e,e,s,s,w,w,p,w,o,p,k,s,u\n")

    line = input("Row> ").strip()

    # Split the input line into individual values
    parts = [p.strip() for p in line.split(",")]

    # 4) Validate input length (must be exactly 22 feature values)
    if len(parts) != 22:
        raise ValueError(f"Input must have exactly 22 values (features only). Got {len(parts)}.")

    values = parts

    # 5) Apply the same missing-value handling used in the notebook
    # In this dataset, missing values are encoded as '?' (typically for stalk-root).
    # In the notebook, these were replaced with the category label 'missing'.
    values = ["missing" if v == "?" else v for v in values]

    # 6) Map the 22 input values to the dataset feature names (excluding 'class')
    # This creates a full "feature name -> value" mapping in the original dataset order.
    full_row = dict(zip(COLUMNS[1:], values))

    # 7) Create a one-row DataFrame using the exact features expected by the model
    # The model may not require all dataset columns (e.g., 'veil-type' can be dropped if constant),
    # so the DataFrame is built strictly in the model's expected feature order.
    X_one = pd.DataFrame([[full_row[f] for f in features]], columns=features)

    # 8) Predict and display results
    # Model prediction is expected to be 'e' or 'p'.
    pred = model.predict(X_one)[0]
    pred_text = "Edible" if pred == "e" else "Poisonous"

    print(f"\nPrediction: {pred_text} ({pred})")


if __name__ == "__main__":
    main()
```

Figure 44: Code snippet of the python CLI file.

Running the prediction on terminal:

Navigate to the correct folder where all the files are kept. After that is done, run the command 'python3 filename.py' to run the python file. The file executes and now we are required to input a row of the dataset's feature for prediction.

```
(base) digdarshanbhattarai@Digdarshans-MacBook-Pro Mushroom CLI % python3 mushroom_cli.py

Mushroom Edibility Predictor
Paste 22 comma-separated feature codes (NO class label).
Order (dataset features, excluding class):
cap-shape, cap-surface, cap-color, bruises, odor, gill-attachment, gill-spacing, gill-size, gill-color, stalk-shape, stalk-root, stalk-surface-above-ring, stalk-surface-below-ring, stalk-color-above-ring,
 stalk-color-below-ring, veil-type, veil-color, ring-number, ring-type, spore-print-color, population, habitat

Example (22 values):
x,s,n,t,p,f,c,n,k,e,e,s,s,w,w,p,w,o,p,k,s,u

Row>
```

Figure 45: Running the prediction on terminal.

Next, open the dataset in a text editor and copy the 22 features, except the first ones as they are the classes for edible or poisonous.

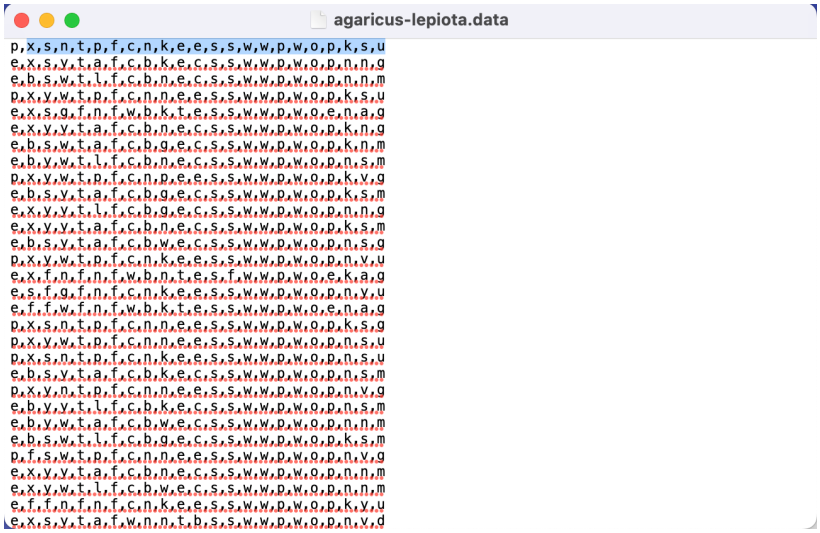

Figure 46: Copying row of features to predict.

Then after, paste the copied row and prediction is made, which is correct.

```
Row> x,s,n,t,p,f,c,n,k,e,e,s,s,w,w,p,w,o,p,k,s,u

Prediction: Poisonous (p)
(base) digdarshanbhattarai@Digdarshans-MacBook-Pro Mushroom CLI %
```

Figure 47: Correct prediction done on given features row.

Detailed Code Pseudocode can be found in APPENDIX 2

7. Conclusion

7.1 Analysis of the Work

This project was based on mushroom edibility prediction as a supervised binary classification problem using the UCI Agaricus-Lepiota dataset (categorical attributes with information about the morphology and the context of a mushroom). The final implementation follows a leakage safe workflow: data cleaning (dealing with the ? missing marker), removing constant feature 'veil-type', stratified train-test split, and model training with scikit-learn pipelines that include one-hot encoding inside cross-validation. Three classifiers were optimised through GridSearchCV (Logistic Regression, Bernoulli Naive Bayes, Random Forest) with a safety-oriented impairment of the general classifier focussing on Recall for the poisonous class (p) followed by testing the classifiers on the held-out test data set (Accuracy, Precision(p), Recall(p) and F1(p)).

Detailed interpretation of metrics:

- Accuracy is a measure of the correctness of both edible and poisonous mushrooms. Logistic Regression and Random Forest gave Accuracy = 1.0000, and the Accuracy of Naive Bayes was 0.9582.
- Precision(p) indicates the reliability of poisonous predictions (the frequency of "poisonous" predictions that are poisonous). Logistic Regression and Random Forest resulted in Precision(p) = 1.0000 while Naive Bayes resulted in Precision(p) = 0.9877 and this represents very little false alarms.
- Recall(p) This is a measure of how many really poisonous mushrooms are distinguished from the non-poisonous ones (really important for safety purposes, since a poisonous mushroom that was predicted as edible is the worst mistake possible). Logistic regression and Random Forest had the Recall(p) = 1.0000, but Naive Bayes had the Recall(p) = 0.9246, so it missed a non-trivial portion of the poisonous cases.
- F1(p) summarises the measure of Precision(p) and Recall(p). Logistic Regression and Random Forest: F1 (p) = 1.0000 Naive Bayesian: F1 (p) = 0.9551 Strong precision, less strong recall. Critical interpretation: As Precision(p) value for all three models are high (especially the value is high for Naive Bayes, 0.9877), the system is effective to avoid false positive (falsely alerting that an edible mushroom is poisonous). However, the more important and error-critical is false negatives (predict poisonous as edible). In this respect the difference is decisive: Naive Bayes shows a low Recall(p) of 0.9246, while Logistic Regression and Random Forest show perfect Recall(p) on the test split, and, thus, have the strongest safety profile under the deemed evaluation protocol.

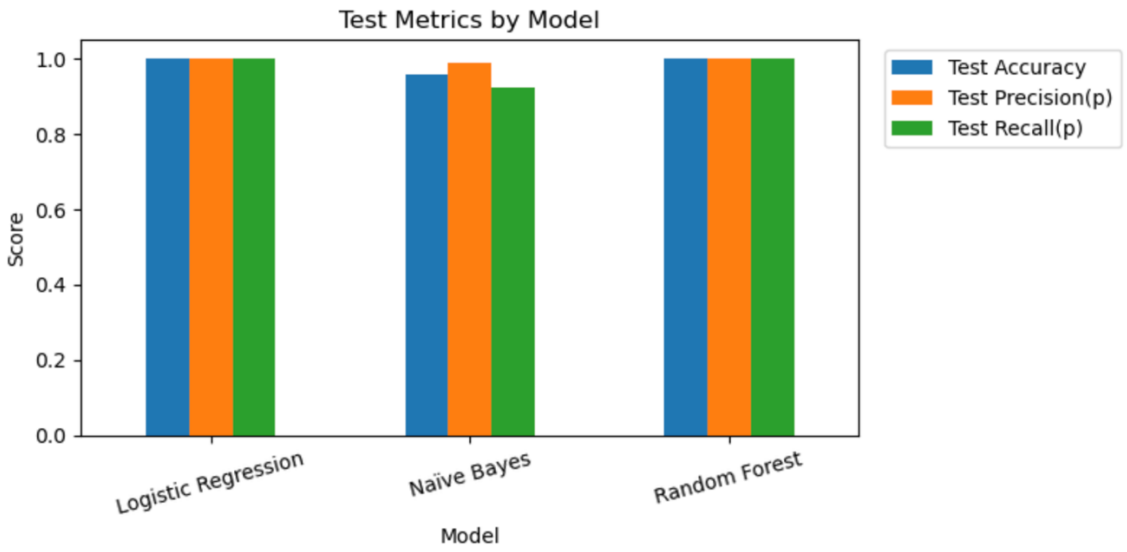

Figure 48: Bar Graph Showing Test Metrics by Model.

7.2 How the Solution Helps to Address Real-World Problems

From a more practical perspective, it is important to note that this is a decision-support mechanism rather than a substitute for human decision-making or laboratory identification. The evaluation framework for this project is very specific about the nature of poisonous identification as a safety-critical problem by optimizing Recall(p), where the worst-case error is a poisonous fungus being classified as edible.

However, the almost perfect performance exhibited by both Logistic Regression and Random Forest should be considered carefully. This UCI dataset can be considered a controlled benchmark dataset, in that it has organized categorical inputs, possibly failing to reflect real-world data collection nuances, in that data can be ambiguous, data points can be inaccurately reported, new categories can emerge, or features might be missing. Thus, the performance reported essentially reflects the ability within the specified feature space. The real-world reliability of the system depends upon the ability of the end-users to correctly assign feature codes.

This is what motivated the design of CLI: it respects the ordering of features in the dataset and is based on the expected features from the trained pipeline.

7.3 Future Work

Improvements in the future should focus on improving robustness and interpretability. Specifically:

- **External Validation:** The trained pipe needs to be tested on new data points such as new mushroom species data entries and on data that is either fake or noisy.
- **Handling uncertainties:** Enhance the command-line interface and the evaluation framework to handle partial inputs where certain features are unknown. This should incorporate methods that assess the impact of missing key attributes on performance.
- **Calibration and risk-aware output:** Add probability calibration and risk awareness in the output policy. For instance, when the confidence level is too low, the model should issue a warning or abstain from making a decision that could be harmful.
- **Model improvement:** Investigate more varied algorithms (such as gradient boosting), and pay more attention to hyperparameter optimization, while again focusing on Recall (p) as a measure of model safety.

8. References

Bibliography

Fernandez, M., 2001. *Publication.* [Online] Available at: https://www.researchgate.net/publication/265266173_Minimal_Decision_Rules_Based_on_the_A_Priori_Algorithm
[Accessed 16 December 2025].

GeeksForGeeks, 2025. *Machine Learning.* [Online] Available at: https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/
[Accessed 11 December 2025].

Nepal Journals Online, 2022. *Article.* [Online] Available at: https://www.nepjol.info/index.php/njmathsci/article/view/44130
[Accessed 11 December 2025].

Paudel, Bhatta, 2022. *Home.* [Online] Available at: https://www.nepjol.info/index.php/njmathsci/article/view/44130
[Accessed 16 December 2025].

Poudel, Bhatta, 2022. *Home.* [Online] Available at: https://www.nepjol.info/index.php/njmathsci/article/view/44130
[Accessed 16 December 2025].

ResearchGate, 2025. *Figure.* [Online] Available at: https://www.researchgate.net/figure/Similar-looking-mushrooms-are-hard-to-differentiate-a-A-picture-of-Chlorophyllum_fig1_382904445
[Accessed 10 December 2025].

ResearchGate, 2025. *Publication.* [Online] Available at: https://www.researchgate.net/publication/393698893_Analysis_of_the_Effectiveness_of_Traditional_and_Ensemble_Machine_Learning_Models_for_Mushroom_Classification
[Accessed 12 December 2025].

Sulistianingsih & Martono, 2025. *Home.* [Online] Available at: https://jurnal.ubhinus.ac.id/index.php/J-INTECH/article/view/1851/1020 [Accessed 16 December 2025].

Sulistianingsih & Martono, 2025. *Home.* [Online] Available at: https://jurnal.ubhinus.ac.id/index.php/J-INTECH/article/view/1851/1020 [Accessed 16 December 2025].

UCI ML Repository, 1987. *dataset.* [Online] Available at: https://archive.ics.uci.edu/dataset/73/mushroom [Accessed 11 December 2025].

UCI, 2025. *Home.* [Online] Available at: https://archive.ics.uci.edu/ [Accessed 10 December 2025].

Verma, A., 2025. *Solutions.* [Online] Available at: https://medium.com/@amitvsolutions/supervised-machine-learning-decoded-from-forecasts-to-decisions-4e57c6b1a0c4 [Accessed 11 December 2025].

Verma, A., 2025. *Solutions.* [Online] Available at: https://medium.com/@amitvsolutions/supervised-machine-learning-decoded-from-forecasts-to-decisions-4e57c6b1a0c4 [Accessed 11 December 2025].

APPENDIX

APPENDIX 1: Dataset Feature Description Table

Table 4: Description of the dataset

Feature Name	**Description**
class	Edibility status (**e or p**)
cap-shape	Shape of mushroom cap
cap-surface	Surface texture of the cap
cap-color	Color of the cap
bruises?	If mushroom bruises or not when pressed
odor	Smell
gill-attachment	How the gills are attached to the stalk
gill-spacing	Space between gills
gill-size	Size/width of gills
gill-color	Color of the gills
stalk-shape	Shape of stalk
stalk-root	Shape of the stalk root
stalk-surface-above-ring	Texture of stalk surface above the ring
stalk-surface-below-ring	Texture of stalk surface below the ring

Feature Name	**Description**
stalk-color-above-ring	Color of the stalk above the ring
stalk-color-below-ring	Color of the stalk below the ring
veil-type	Veil type covering the mushroom
veil-color	Color of the veil
ring-number	Number of rings present on the stalk
ring-type	Appearance of the ring
spore-print-color	Colour of the spore print
population	Density of population
habitat	Environment where they grow

Return to the previous page

APPENDIX 2: Pseudocode for CLI

INPUT: 22 comma separated categorical features

OUTPUT: Edible or Poisonous label

START

1. LOAD the trained pipeline from mushroom_model.joblib

2. READ the feature names expected by the model

3. DISPLAY the required input order (22 features) and an example

4. READ one input line from the user

5. SPLIT the line by commas into a list of values

6. If the number of values is not 22, stop with an error

7. REPLACE ‘ ?’ value with ‘missing’

8. MAP the 22 values to the 22 dataset feature names (excluding class)

9. CREATE a one-row table using only the model’s expected feature order

10. PREDICT the class using the loaded model

10. CONVERT the prediction to text: e → Edible, p → Poisonous

11. PRINT the final prediction

END

Return to the previous page